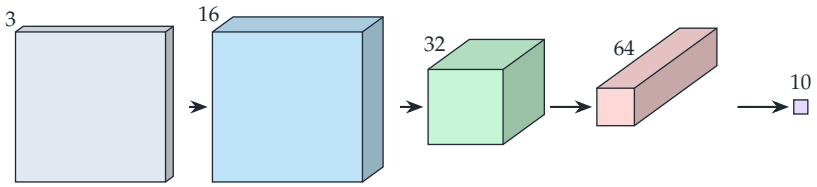
ResNet-20 on CIFAR-10, End to End

A shape-by-shape refresher of the `resnet-cifar10` repository



Lasfer Mohammed Djawed

`github.com/fyl3x/resnet-cifar10`

October 2026

This document explains every step of the project: raw bytes, augmentation, normalisation, batching, every convolution, normalisation and shortcut, the logits and the loss, backpropagation through each operation, the optimiser, the schedule, evaluation and serving. It gives the shape of every tensor and the formula behind every number. The repository code is the source of truth: each claim about "what the code does" was checked against `cifar_resnet/`, `scripts/` and `service/`, and the numerical examples were produced by running the trained checkpoint `model/resnet20.pt` on `samples/test0.png`. Reference: K. He, X. Zhang, S. Ren, J. Sun, *Deep Residual Learning for Image Recognition*, CVPR 2016, arXiv:1512.03385.

Contents

1 The whole project on one page **2**

1.1 What the project is . . . 2
1.2 Pipeline at a glance . . . 2
1.3 Code map . . . 2
1.4 The configuration that every number in this document assumes . . . 3
1.5 Notation . . . 3

2 Data: from bytes on disk to a batch tensor **3**

2.1 Raw CIFAR-10 as torchvision hands it over . . . 3
2.2 Train / validation split . . . 4
2.3 Per-pixel mean . . . 4
2.4 Augmentation (training only) . . . 5
2.5 Preprocessing: the function shared by training and serving . . . 6
2.6 Dataset and batching . . . 7

3 The mathematical toolkit, one operation at a time **8**

3.1 Matrix multiplication and how shapes align . . . 8
3.2 The fully connected (linear) layer . . . 9
3.3 Tensors and the meaning of every axis . . . 10
3.4 2-D convolution . . . 10
3.5 Convolution *is* a matrix multiplication (im2col) . . . 12
3.6 Batch normalisation . . . 13
3.7 ReLU . . . 14
3.8 Global average pooling and flatten . . . 15
3.9 Logits, softmax, cross-entropy and class selection . . . 15

4 One residual block in full detail **16**

4.1 The idea: learn a correction, not a whole mapping . . . 16
4.2 The code . . . 17
4.3 The block as equations . . . 17
4.4 Identity block: 16 channels in, 16 out, stride 1 . . . 18
4.5 Downsampling block: 16 channels in, 32 out, stride 2 . . . 19
4.6 The plain block . . . 20
4.7 Why the shortcut helps optimisation (the one-line gradient argument) . . . 20

4.8 Activations in a real block 21

5 The complete ResNet-20 and its shape flow 21

5.1 The $6n + 2$ family 21

5.2 The shape flow, drawn 22

5.3 The full layer table 22

5.4 Initialisation 24

5.5 Memory of one forward pass 24

6 A complete forward pass with real numbers 25

6.1 Input 25

6.2 One output pixel of the stem, computed by hand 25

6.3 BatchNorm in eval mode on real parameters 26

6.4 Statistics through the whole network 26

6.5 From pooled features to a decision 27

6.6 An independent check of every formula in this document 27

7 Back-propagation through every operation 27

7.1 The rule that organises everything 27

7.2 Cross-entropy with softmax: the gradient on the logits 28

7.3 Linear layer 29

7.4 Flatten and global average pooling 29

7.5 ReLU 29

7.6 The residual addition and the option-A shortcut 29

7.7 Batch normalisation (train mode) 30

7.8 Convolution 31

7.9 One whole block backwards 32

8 Optimisation: SGD, the schedule and the training loop 32

8.1 The optimiser 32

8.2 Weight decay 33

8.3 The learning-rate schedule 34

8.4 The training loop 35

8.5 Seeds, checkpoints and the run script 36

8.6 Sanity checks that were run before the full training 37

9 Evaluation: eval mode, metrics and results 37

9.1 What changes in eval mode 37

9.2 Confusion matrix 38
9.3 Precision, recall, F1 38
9.4 Calibration and expected calibration error 38
9.5 Headline result 40
9.6 Depth ablation: plain vs. residual 40

10 Inference and serving **42**

A One-page cheat sheet **44**

B Things this project does *not* contain **45**

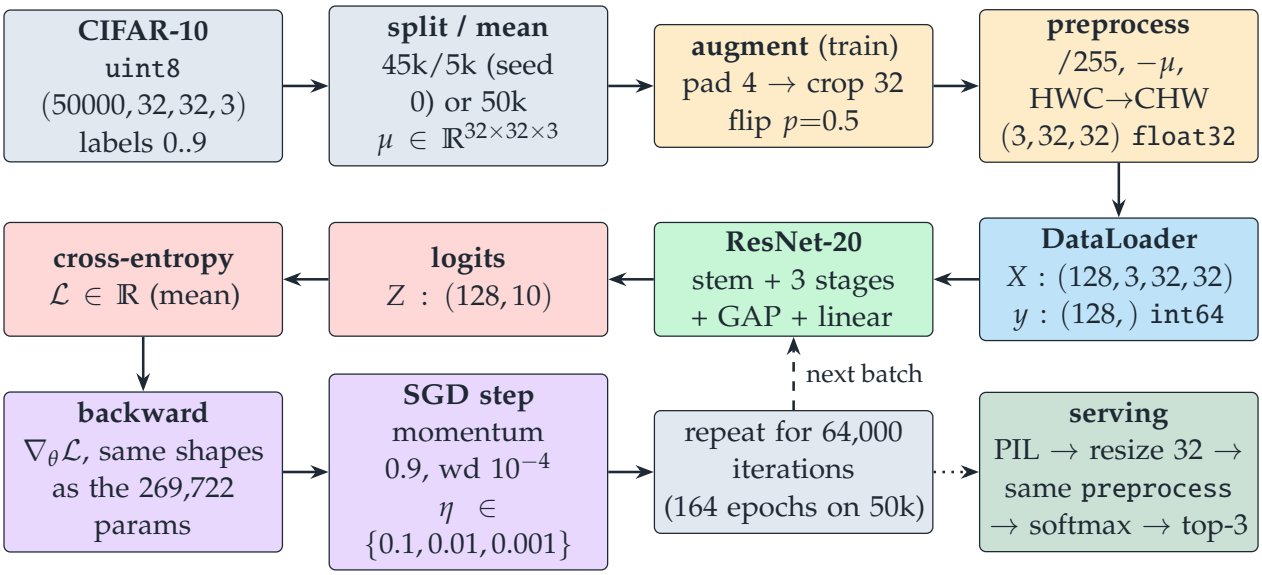
C How this document was verified **45**

1 The whole project on one page

1.1 What the project is

The repository `fyl3x/resnet-cifar10` reproduces the CIFAR-10 ResNet of He et al. (2015), *Deep Residual Learning for Image Recognition*, Section 4.2. The network is the $6n + 2$ family with $n = 3$ (ResNet-20) as the headline model and $n = 9$ (ResNet-56) for the depth ablation. Everything is written in plain PyTorch: the blocks, the He initialisation, the augmentation, the iteration-based learning-rate schedule, the training loop and the metrics. The final model reaches **8.42% test error** (paper: 8.75%, both single runs) and is served by a FastAPI app inside a CPU Docker image.

1.2 Pipeline at a glance



1.3 Code map

File	What it contains	Explained in
`cifar_resnet/data.py`	`split_indices`, `compute_pixel_mean`, `augment`, `preprocess`, `CIFARDataset`	Section 2
`cifar_resnet/model.py`	`BasicBlock` (residual or plain), `ResNetCIFAR` ($6n+2$, He init)	Sections 4 and 5
`cifar_resnet/train.py`	`train_step`, `train_one_epoch`, `evaluate`, `lr_at`, `set_lr`, checkpoints	Section 8
`cifar_resnet/evaluate.py`	`predict` (softmax probs), confusion matrix, precision/recall/F1, ECE	Section 9
`scripts/train_resnet.py`	CLI: `-n -plain -seed -full -name -max-iters`; builds data, model, SGD, logs CSV	Section 8
`scripts/eval_test.py`, `scripts/make_report.py`, `scripts/make_ablation.py`	test-set evaluation, figures, ablation table	Section 9
`service/app.py`	FastAPI: `GET /health`, `POST /predict`	Section 10
`scripts/check_*.py`, `overfit_batch.py`	the development checks run at each stage	throughout

1.4 The configuration that every number in this document assumes

Quantity	Value in this repo	Where it is set
Depth parameter n	3 (ResNet-20); 9 for ResNet-56	`-n`, `ResNetCIFAR(n)`
Stage widths	$(16, 32, 64)$	`widths=(16,32,64)`
Residual or plain	residual; `-plain` removes the shortcut	`BasicBlock(residual=...)`
Shortcut	option A: stride-2 subsample + zero channel padding	`BasicBlock.forward`
Classes	10	`num_classes=10`
Batch size N	128	`DataLoader(batch_size=128)`
Optimiser	SGD, momentum $\mu = 0.9$, weight decay $\lambda = 10^{-4}$	`train_resnet.py`
Learning rate η	0.1 / 0.01 / 0.001, switching at iterations 32k / 48k	`lr_at`
Iterations	64,000	`-max-iters`
Loss	`nn.CrossEntropyLoss()` (mean over batch)	`train_resnet.py`
Initialisation	He normal, $\sigma = \sqrt{2/\text{fan_in}}$, convs only	`_init_weights`
Final training set	all 50,000 images (`-full`)	`train_resnet.py`
Development split	45,000 / 5,000, split seed fixed to 0	`split_indices(50000,5000,0)`
Parameters	269,722 (ResNet-20), 853,018 (ResNet-56)	counted from the model

1.5 Notation

N	batch size (128)	$\mathcal{W}$	4-D conv weight $(C_{\text{out}}, C_{\text{in}}, K, K)$
C, H, W	channels, height, width	W (linear)	weight matrix $(10, 64)$
K, S, P	kernel size, stride, padding	γ, β	BN scale and shift, shape $(C,)$
x, X	one input / a batch	μ_c, σ_c^2	BN batch mean and variance of channel c
$\mathcal{F}(x)$	residual branch of a block	z, Z	logits of one image / a batch
s	softmax probabilities	$\mathcal{L}$	mean cross-entropy over the batch
G_T	$\partial\mathcal{L}/\partial T$, *same shape as* T	η, μ, λ	learning rate, momentum, weight decay

Shapes are written in PyTorch order, e.g. $(128, 16, 32, 32)$. In drawings, a box of width W and height H with depth going into the page is one image's feature tensor, and the depth is its channel count. Box colours follow the stage: input, 16 channels, 32 channels, 64 channels, weights, outputs/gradients.

Each section separates the **general idea** (what the mathematics says, independent of this code), **in this repository** (what the code actually does, with sizes), **details and deviations** (where the code differs from a textbook or a reader could be surprised), and **shape checks**.

2 Data: from bytes on disk to a batch tensor

2.1 Raw CIFAR-10 as torchvision hands it over

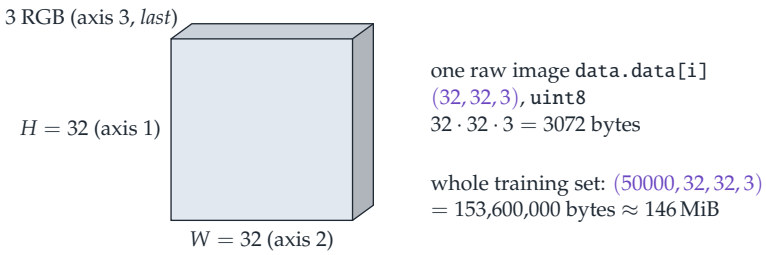
In this repository: `torchvision.datasets.CIFAR10(root="data", train=..., download=True)`

The dataset object exposes two plain attributes, and the repo uses them directly. It never goes through torchvision's transforms or its `__getitem__`:

- `data.data`: a NumPy array of shape $(50000, 32, 32, 3)$ for `train=True` or $(10000, 32, 32, 3)$ for `train=False`, `dtype=uint8`, values 0..255, channel order RGB, layout *HWC*.
- `data.targets`: a Python `list` of 50,000 (or 10,000) integers in 0..9.

Class indices: 0 airplane, 1 automobile, 2 bird, 3 cat, 4 deer, 5 dog, 6 frog, 7 horse, 8 ship, 9 truck.

Training has 5,000 images per class and test has 1,000 per class.



2.2 Train / validation split

```
5  def split_indices(n,n_val,seed):
6      g = torch.Generator()
7      g.manual_seed(seed)
8      result = torch.randperm(n=n,generator=g)
9      validation_idx = result[:n_val]
10     training_idx = result[n_val:]
11     return training_idx,validation_idx
```

General idea

Draw one random permutation of $\{0, \ldots, n-1\}$ and cut it: the first n_{val} indices are validation and the rest are training. The two sets are disjoint and together cover everything. This is checked in `scripts/check_split.py`: 45,000 + 5,000, zero overlap, same seed $\Rightarrow$ same split.

In this repository: two regimes

- **Development** (default, used for all four ablation runs): `split_indices(50000, 5000, 0)` gives 45,000 training and 5,000 validation indices, both `int64` tensors.
- **Final** (`-full`): `train_idx = torch.arange(50000)`, with no validation set. The test set (10,000) is used only once, in `eval_test.py`.

Detail / deviation: the split seed is hard-coded

`train_resnet.py` calls `split_indices(50000,5000,0)` with a literal `0`, not `args.seed`. Changing `-seed` changes the initialisation, the shuffling and the augmentation, but *not* which 5,000 images are held out. This is deliberate: the 45k mean file `pixel_mean_45k_seed0.npy` is only valid for that split.

2.3 Per-pixel mean

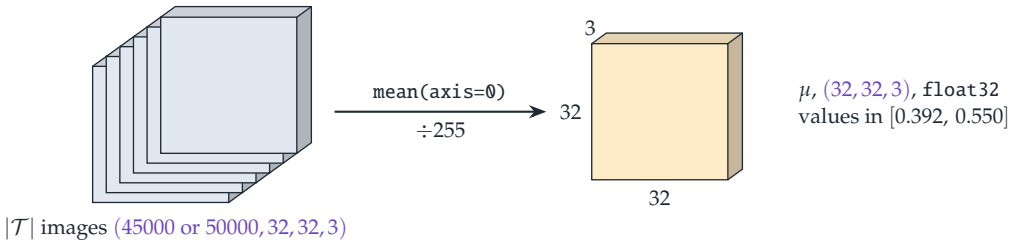
```
13 def compute_pixel_mean(images,train_idx):
14     training_images = images[train_idx.numpy()]
15     floated_images = training_images.astype(np.float32)
16     avg = floated_images.mean(axis=0)
17     avg /= 255
18     return avg
```

General idea: mean over images, not over pixels

With $\mathcal{T}$ the set of training indices,

$$\mu[h,w,c] = \frac{1}{255\,|\mathcal{T}|}\sum_{i\in\mathcal{T}} x_i[h,w,c], \qquad \mu \in \mathbb{R}^{32\times32\times3}.$$

This is a *per-pixel, per-channel* mean: 3,072 separate numbers, an "average image". Contrast it with the more common *per-channel* mean (3 numbers, the average over images *and* positions). The paper says "the per-pixel mean subtracted", and that is what this repo implements.



In this repository: the actual mean file `model/pixel_mean_50k.npy`

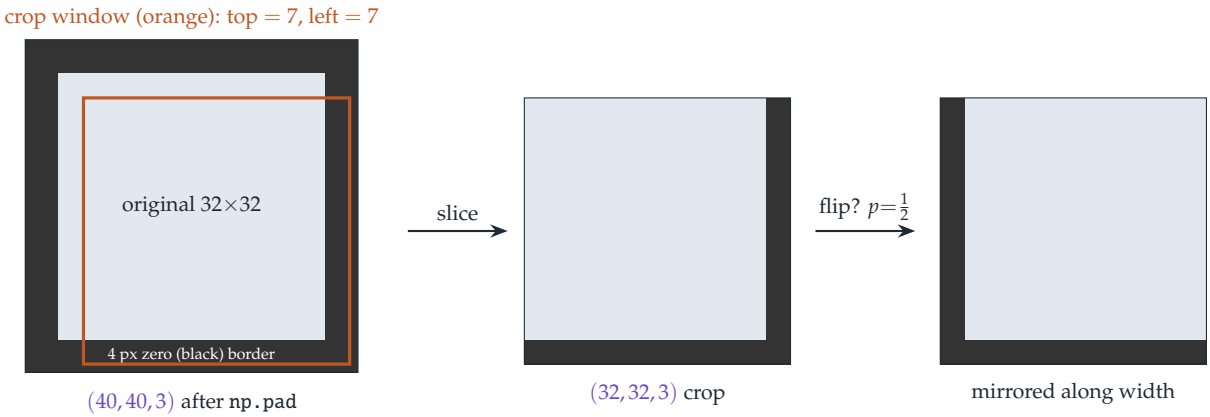
Shape $(32,32,3)$, `float32`, 12,416 bytes on disk (12,288 data + header). Averaged over positions, the three channels are $(0.4914,\ 0.4822,\ 0.4465)$, the familiar CIFAR-10 channel means. Individual entries range from 0.392 to 0.550; for example $\mu[0,0] = (0.513, 0.534, 0.520)$ at the top-left corner and $\mu[16,16] = (0.502, 0.457, 0.421)$ at the centre. The corner is greyer and the centre more reddish, which is where objects usually sit. Two files exist: `pixel_mean_45k_seed0.npy` (development split) and `pixel_mean_50k.npy` (final model and service). The mean is always computed on the raw, *un-augmented* training images.

Detail / deviation: no division by a standard deviation

`preprocess` subtracts the mean and does not divide by any standard deviation. The paper does not mention one either, and BN right after the stem rescales the activations anyway. Inputs therefore keep the raw $[0,1]$ spread, roughly $[-0.55,\ 0.61]$ after centring.

2.4 Augmentation (training only)

```
20 def augment(image,rng):
21     padded_image = np.pad(image,pad_width=((4, 4), (4, 4), (0, 0)))
22     top_offset = rng.integers(0,9)
23     left_offset = rng.integers(0,9)
24     cropped_image = padded_image[top_offset:top_offset+32, left_offset:left_offset+32, :]
25     if rng.random() < 0.5 :
26         cropped_image = np.ascontiguousarray(np.flip(cropped_image, axis=1))
27     return cropped_image
```



General idea

Random translation by up to ±4 pixels plus a random left–right mirror. The label stays the same, and the network sees a slightly different image every epoch. This is the standard "pad 4, random crop 32, horizontal flip" of the paper.

In this repository: exact behaviour

- `np.pad` with no mode means constant **0**, applied to the `uint8` image, so the border is pure black *before* normalisation.
- `rng.integers(0,9)` draws from $\{0, 1, \ldots, 8\}$ (upper bound exclusive): 9 vertical × 9 horizontal offsets = 81 crop positions, and offset 4 is the un-shifted image. With the flip, that gives up to 162 distinct views of each image.
- `np.flip(axis=1)` reverses the *width* axis of an HWC array, which is a horizontal mirror. `np.flip` returns a negative-stride view, so `np.ascontiguousarray` copies it into normal memory.
- Three random draws per image, in order: top offset, left offset, flip coin.
- `CIFARDataset` owns *one* `np.random.default_rng(seed)` created in `__init__`, used only when `train=True`. Its stream is consumed in the order the sampler requests images, so the whole augmentation sequence is reproducible given the seed.

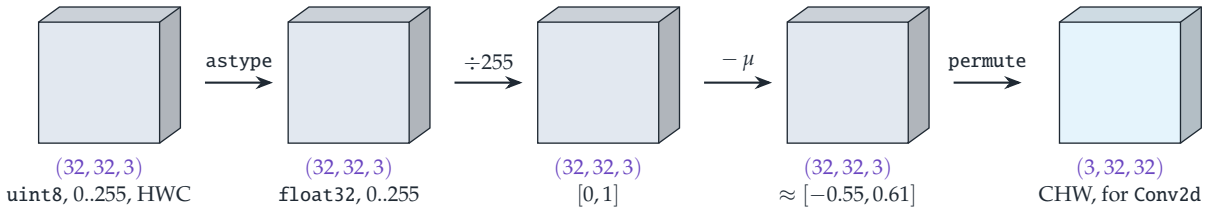
Detail / deviation: two subtle consequences

(1) Padding is not zero after preprocessing. Pixels in the black border become $0/255 - \mu[h, w, c] \approx (-0.49, -0.48, -0.45)$, not 0. The network sees "black", which is a legitimate image colour. It is *not* the same as the zero padding inside the convolutions. Common pipelines that pad a PIL image before normalising behave the same way.
(2) The RNG would be duplicated by worker processes. With `DataLoader(num_workers>0)`, each worker gets a copy of the same `rng` state, so different workers would produce identical augmentation sequences. The repo uses the default `num_workers=0` (single process), so this does not happen. It is something to remember before "speeding up" the loader.

2.5 Preprocessing: the function shared by training and serving

```
29 def preprocess(image,mean):
30     image = image.astype(np.float32)
31     image = image / 255
32     image = image - mean
33     tensored_image = torch.tensor(image)
34     tensored_image = tensored_image.permute(2,0,1).contiguous()
```

```
35     return tensored_image
```



General idea: why the permute

NumPy, PIL and torchvision store images channel-last (HWC). `nn.Conv2d` expects channel-first (CHW, and NCHW for a batch). `permute(2,0,1)` reorders the axes: new axis 0 = old axis 2 (channels), new 1 = old 0 (rows), new 2 = old 1 (columns). Mathematically, $X_{\text{out}}[c,h,w] = X_{\text{in}}[h,w,c]$. `permute` only changes strides, and `.contiguous()` materialises a copy laid out in the new order.

In this repository: broadcasting in image $-\mu$

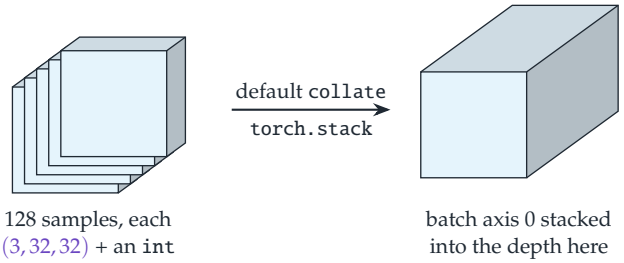
Both operands are $(32,32,3)$, so the subtraction is element-wise with no broadcasting: each pixel subtracts *its own* mean. Worked example from `samples/test0.png` (a cat): raw top-left pixel $(158,112,49)$, so $(0.6196,0.4392,0.1922) - (0.5126,0.5336,0.5198) = (0.1070,-0.0943,-0.3277)$, which is exactly `x[0,:,0,0]` in the model input. Over the whole image the values lie in $[-0.438, 0.591]$ with mean -0.048.

2.6 Dataset and batching

```
37 class CIFARDataset(torch.utils.data.Dataset):
38     ...
39     def __getitem__(self, i):
40         idx = int(self.indices[i])
41         image, label = self.images[idx], self.labels[idx]
42         if self.train:
43             image = augment(image=image,rng=self.rng)
44         return preprocess(image, self.mean), label
```

General idea: indirection through `indices`

The dataset never copies the images. It keeps the full array plus an index list, and position i in the dataset maps to raw image `indices[i]`. The same object therefore serves the 45k split, the 5k split, the full 50k and the 10k test set.



X: $(128,3,32,32)$, `float32`, 1.5 MiB

y: (128), `int64` (from Python `int`s)

In this repository: loaders and the number of batches

- Training: `DataLoader(batch_size=128, shuffle=True, generator=g)` with `g` seeded by `-seed`. A new random order is drawn from `g` every epoch. `drop_last` is `False`, so the remainder forms a short final batch.
- Validation and test: `shuffle=False`, `train=False`, so there is no augmentation.

set	images	full batches	last batch	batches per pass
final training (50k)	50,000	390	80	**391**
development training (45k)	45,000	351	72	**352**
validation	5,000	39	8	40
test	10,000	78	16	79

These numbers fix the epoch arithmetic of Section 8.3: one epoch on 50k is 391 *iterations*, and the schedule counts iterations, not epochs.

3 The mathematical toolkit, one operation at a time

Every layer in ResNet-20 is one of six operations: matrix multiplication (inside the linear layer), 2-D convolution, batch normalisation, ReLU, global average pooling, and the softmax/cross-entropy pair at the end. This section treats each one on its own, first in general, then with the exact sizes this repository uses. Sections 4 and 5 then just compose them.

3.1 Matrix multiplication and how shapes align

General idea: the inner dimensions must match and disappear

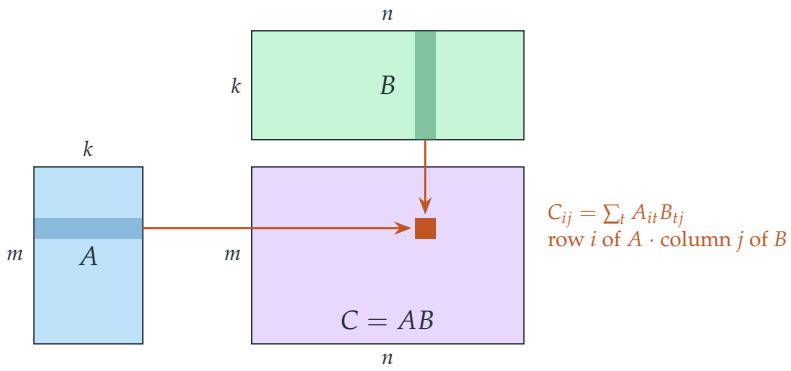
For $A \in \mathbb{R}^{m\times k}$ and $B \in \mathbb{R}^{k\times n}$, the product $C = AB \in \mathbb{R}^{m\times n}$ is

$$C_{ij} \;=\; \sum_{t=1}^{k} A_{it}\, B_{tj}, \qquad i = 1..m,\; j = 1..n.$$

Entry C_{ij} is the dot product of *row i of A* with *column j of B*, so both must have length k. The shared (inner) dimension k is summed away; the outer dimensions m and n survive:

$$\underbrace{(m \times \cancel{k})}_{A} \cdot \underbrace{(\cancel{k} \times n)}_{B} \;=\; \underbrace{(m \times n)}_{C}, \qquad \text{cost} = m\,k\,n \text{ multiply–adds.}$$

The picture below is the one to keep in your head: put B above C and A to its left. Then every cell of C sits at the crossing of one row of A and one column of B.



In this repository: where matrix multiplication appears

Explicitly, only once: the final `nn.Linear(64, 10)`. Implicitly, every convolution is a matrix multiplication after the *im2col* rearrangement (Section 3.5). Batch normalisation, ReLU and pooling are not matrix multiplications.

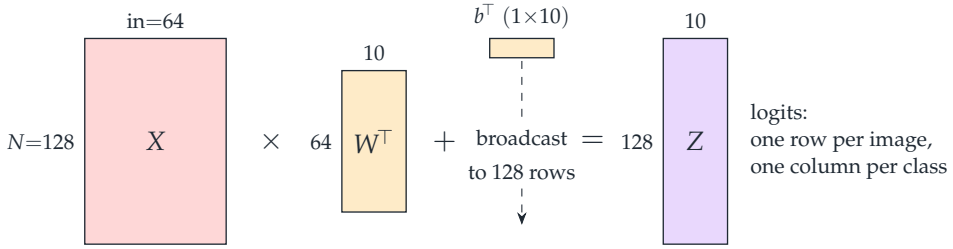
3.2 The fully connected (linear) layer

General idea: affine map, PyTorch convention

PyTorch stores the weight of `nn.Linear(in, out)` as $W \in \mathbb{R}^{\text{out}\times\text{in}}$ and computes, for a batch $X \in \mathbb{R}^{N\times\text{in}}$ (one example per row),

$$Z \;=\; X\,W^{\top} + \mathbf{1}_N\, b^{\top}, \qquad Z \in \mathbb{R}^{N\times\text{out}}, \quad b \in \mathbb{R}^{\text{out}}.$$

Each row $z_n = Wx_n + b$ is the textbook "$Wx + b$"; stacking examples as rows is what turns it into $XW^{\top}$. The bias row is *broadcast*: the same b is added to every one of the N rows.



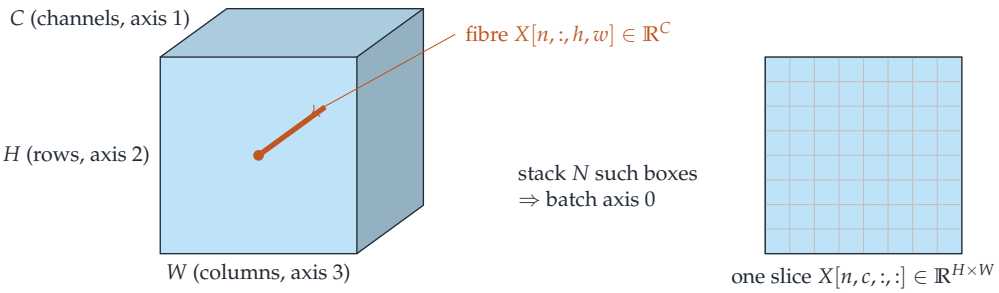
In this repository: `self.linear = nn.Linear(64, 10)`

$W \in \mathbb{R}^{10\times 64}$ (640 numbers) and $b \in \mathbb{R}^{10}$ (10 numbers): 650 parameters, the only layer with a bias in the whole network. Its input is the 64-dimensional pooled feature vector of each image, its output the 10 logits. This is the only fully connected ("feed-forward") layer in the model: there is no hidden MLP head, no dropout.

3.3 Tensors and the meaning of every axis

General idea: NCHW

A batch of images is a 4-D tensor $X \in \mathbb{R}^{N\times C\times H\times W}$. I draw one image ($N$ fixed) as a box: width W to the right, height H upward, and *depth* C going into the page. Each depth slice $X[n, c, :, :]$ is one $H \times W$ *feature map*; each depth fibre $X[n, :, h, w]$ is the C-vector describing location (h, w).



In this repository: axis meaning at each point of the pipeline

Where	Layout	dtype	Meaning
`data.data` (torchvision)	$(50000, 32, 32, 3)$	`uint8`	images, *HWC* order, values 0..255
inside `augment`	$(32, 32, 3) \to (40, 40, 3) \to (32, 32, 3)$	`uint8`	one image, still HWC
output of `preprocess`	$(3, 32, 32)$	`float32`	CHW, mean-subtracted
a training batch	$(128, 3, 32, 32)$	`float32`	NCHW, what the model eats
labels batch	$(128,)$	`int64`	class index 0..9 per image
logits	$(128, 10)$	`float32`	one score per (image, class)

3.4 2-D convolution

General idea: what `nn.Conv2d` computes

A convolution layer with C_{in} input channels, C_{out} output channels, kernel size K, stride S and zero padding P has a weight tensor

$$\mathcal{W} \in \mathbb{R}^{C_{\text{out}}\times C_{\text{in}}\times K\times K}.$$

Pad the input with P zeros on every side, $\tilde{X}[n, c, a, b] = X[n, c, a - P, b - P]$ (zero outside the image). Then, with no bias,

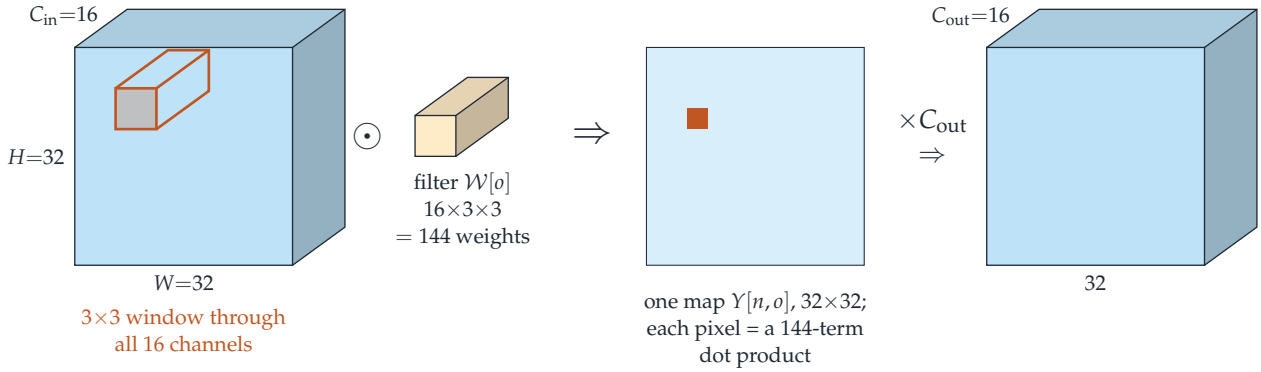
$$Y[n, o, i, j] = \sum_{c=0}^{C_{\text{in}}-1} \sum_{u=0}^{K-1} \sum_{v=0}^{K-1} \mathcal{W}[o, c, u, v]\ \tilde{X}[n,\ c,\ Si + u,\ Sj + v]$$

Strictly this is a cross-correlation (the kernel is not flipped). Deep-learning libraries call it convolution anyway. Since the weights are learned, the difference does not matter.
Three facts to keep:

1. **Each filter $\mathcal{W}[o]$ is a $C_{\text{in}} \times K \times K$ block**, as deep as the input. It sums over *all* input channels and produces *one* 2-D map.

2. **The number of filters is the output depth.** C_{out} filters give C_{out} maps, stacked along axis 1. This is how depth "is added".
3. **Spatial size** follows from the window positions that fit:

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2P - K}{S} \right\rfloor + 1, \qquad W_{\text{out}} \text{ likewise.}$$

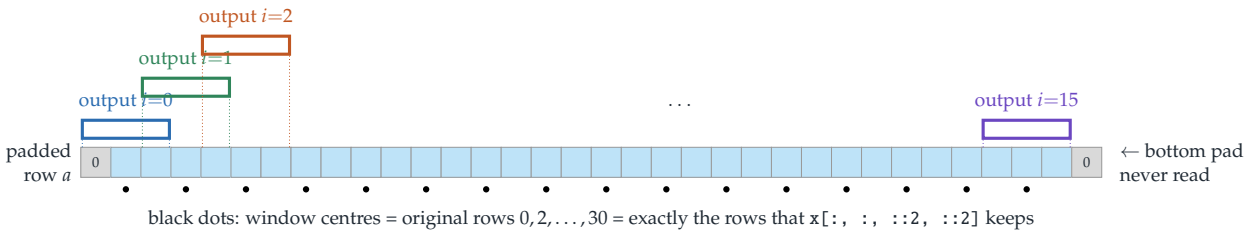


Worked output-size examples (all convolutions in this repo use $K = 3$, $P = 1$).

$$S = 1: \quad H_{\text{out}} = \left\lfloor \tfrac{32+2-3}{1} \right\rfloor + 1 = 31 + 1 = 32 \qquad \text{("same" padding: size preserved)}$$
$$S = 1: \quad \left\lfloor \tfrac{16+2-3}{1} \right\rfloor + 1 = 16, \qquad \left\lfloor \tfrac{8+2-3}{1} \right\rfloor + 1 = 8$$
$$S = 2: \quad H_{\text{out}} = \left\lfloor \tfrac{32+2-3}{2} \right\rfloor + 1 = \lfloor 15.5 \rfloor + 1 = 16 \qquad \text{(halved)}$$
$$S = 2: \quad \left\lfloor \tfrac{16+2-3}{2} \right\rfloor + 1 = \lfloor 7.5 \rfloor + 1 = 8.$$

Detail / deviation: what stride 2 actually reads

With $K = 3, P = 1, S = 2$ on a 32-row input, output row i reads padded rows $2i, 2i{+}1, 2i{+}2$, i.e. original rows $2i - 1$, $2i$, $2i + 1$. The window is *centred on even input rows* $0, 2, \ldots, 30$. The last output ($i = 15$) reads original rows 29–31, so the padding row added at the bottom (and the column added on the right) is never touched. Only the top/left zero padding is used. The option-A shortcut (Section 4.5) samples $x[:, :, :: 2, :: 2]$, the same even positions, so the shortcut and the convolution branch stay spatially aligned.



In this repository: every convolution in the model

All convolutions are `nn.Conv2d(..., kernel_size=3, padding=1, bias=False)`. There are exactly three kinds:

kind	$C_{\text{in}} \to C_{\text{out}}$	S	$H_{\text{in}} \to H_{\text{out}}$	weight shape	#weights
stem	$3 \to 16$	1	$32 \to 32$	$(16, 3, 3, 3)$	432
stage 1 (all 6)	$16 \to 16$	1	$32 \to 32$	$(16, 16, 3, 3)$	2,304
stage 2, first	$16 \to 32$	2	$32 \to 16$	$(32, 16, 3, 3)$	4,608
stage 2, other 5	$32 \to 32$	1	$16 \to 16$	$(32, 32, 3, 3)$	9,216
stage 3, first	$32 \to 64$	2	$16 \to 8$	$(64, 32, 3, 3)$	18,432
stage 3, other 5	$64 \to 64$	1	$8 \to 8$	$(64, 64, 3, 3)$	36,864

There is no 1×1 projection convolution anywhere. The shortcut has no parameters (Section 4.5). Depth changes in only three places: the stem ($3 \to 16$) and the first convolution of stages 2 and 3 ($16 \to 32$, $32 \to 64$), and each of those last two coincides with a stride-2 halving of the spatial size.

Detail / deviation: why `bias=False`

Every convolution is immediately followed by batch normalisation, which subtracts the per-channel mean. A per-channel conv bias would be subtracted right back out, so it would be a wasted parameter. BN's own shift β plays the role of the bias.

3.5 Convolution *is* a matrix multiplication (im2col)

General idea: unfold, multiply, fold

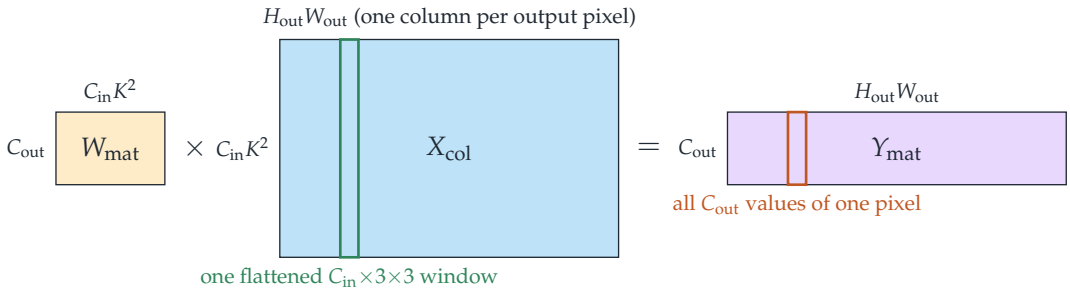
For one image, cut out every $C_{\text{in}} \times K \times K$ window that the filter visits, flatten each to a column of length $C_{\text{in}}K^2$, and line the columns up:

$$X_{\text{col}} \in \mathbb{R}^{(C_{\text{in}}K^2)\times(H_{\text{out}}W_{\text{out}})}.$$

Flatten each filter into a row: $W_{\text{mat}} = \mathcal{W}.\text{reshape}(C_{\text{out}}, -1) \in \mathbb{R}^{C_{\text{out}}\times(C_{\text{in}}K^2)}$. Then the whole convolution is one matrix product

$$\underbrace{Y_{\text{mat}}}_{C_{\text{out}}\times H_{\text{out}}W_{\text{out}}} = \underbrace{W_{\text{mat}}}_{C_{\text{out}}\times C_{\text{in}}K^2} \underbrace{X_{\text{col}}}_{C_{\text{in}}K^2\times H_{\text{out}}W_{\text{out}}},$$

and reshaping Y_{mat} to $(C_{\text{out}}, H_{\text{out}}, W_{\text{out}})$ gives the output maps. The inner dimension $C_{\text{in}}K^2$ is "one window", and summing over it is exactly the triple sum in the boxed formula above.



Shape check: im2col sizes for this network, per image

layer	W_{mat}	X_{col}	$Y_{\text{mat}} \to$ reshaped
stem	16×27	27×1024	$16 \times 1024 \to (16, 32, 32)$
stage-1 conv	16×144	144×1024	$16 \times 1024 \to (16, 32, 32)$
stage-2 first conv (S=2)	32×144	144×256	$32 \times 256 \to (32, 16, 16)$
stage-2 conv	32×288	288×256	$32 \times 256 \to (32, 16, 16)$
stage-3 first conv (S=2)	64×288	288×64	$64 \times 64 \to (64, 8, 8)$
stage-3 conv	64×576	576×64	$64 \times 64 \to (64, 8, 8)$

For the stem: $27 = 3 \cdot 3 \cdot 3$ (RGB $\times$ 3×3 window) and $1024 = 32 \cdot 32$ output pixels. For a batch, do this for each of the N images (or concatenate their columns: X_{col} becomes $C_{\text{in}}K^2 \times NH_{\text{out}}W_{\text{out}}$).

Detail / deviation: implementation vs. mathematics

PyTorch on a GPU uses cuDNN, which picks among implicit-GEMM, Winograd and FFT algorithms and does not literally build X_{col}. The result is the same function up to floating-point rounding. Since the repo does not set `torch.backends.cudnn.deterministic`, two GPU runs with the same seed can differ in the last bits.

Cost of a convolution. Each output entry is a dot product of length $C_{\text{in}}K^2$, and there are $C_{\text{out}}H_{\text{out}}W_{\text{out}}$ entries, so a conv costs $C_{\text{out}}C_{\text{in}}K^2H_{\text{out}}W_{\text{out}}$ multiply–adds (MACs) per image. A 16→16 conv at 32×32 costs $16 \cdot 16 \cdot 9 \cdot 1024 = 2{,}359{,}296$ MACs. A 32→32 conv at 16×16 costs $32 \cdot 32 \cdot 9 \cdot 256$, *exactly the same number*, and so does 64→64 at 8×8. This is the ResNet design rule: *when the feature map is halved, double the number of filters*, which keeps the time per layer constant.

3.6 Batch normalisation

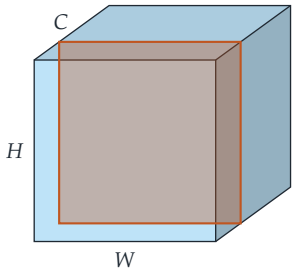
General idea: normalise each channel over the batch, then re-scale

For a 4-D activation $A \in \mathbb{R}^{N \times C \times H \times W}$, `BatchNorm2d(C)` treats each channel c separately and pools statistics over the $m = N \cdot H \cdot W$ values of that channel:

$$\mu_c = \frac{1}{m} \sum_{n,h,w} A[n,c,h,w], \qquad \sigma_c^2 = \frac{1}{m} \sum_{n,h,w} \big(A[n,c,h,w] - \mu_c\big)^2,$$

$$\hat{A}[n,c,h,w] = \frac{A[n,c,h,w] - \mu_c}{\sqrt{\sigma_c^2 + \varepsilon}}, \qquad Y[n,c,h,w] = \gamma_c \, \hat{A}[n,c,h,w] + \beta_c.$$

$\gamma, \beta \in \mathbb{R}^C$ are learned and are broadcast as shape $(1, C, 1, 1)$. After normalisation each channel has mean 0 and variance ≈ 1 over the batch. γ and β then let the network choose any other scale and offset.



channel c in image n:
$H \cdot W$ values

μ_c, σ_c^2 pool this slice *across all N images*
$\Rightarrow m = N \cdot H \cdot W$ values per statistic

In this repository: PyTorch `BatchNorm2d` defaults, used as-is

$\varepsilon = 10^{-5}$, `momentum` $\alpha = 0.1$, `affine=True` (γ initialised to 1, β to 0), `track_running_stats=True`. Each BN layer therefore has $2C$ parameters and three buffers: `running_mean` $\in \mathbb{R}^C$, `running_var` $\in \mathbb{R}^C$, `num_batches_tracked` (an integer).
In **train mode** (`model.train()`), the batch statistics above are used, and the running buffers are updated after every forward pass:

$$r_\mu \leftarrow (1-\alpha)\, r_\mu + \alpha\, \mu_c, \qquad r_{\sigma^2} \leftarrow (1-\alpha)\, r_{\sigma^2} + \alpha\, \tfrac{m}{m-1}\, \sigma_c^2.$$

Note the asymmetry. The *normalisation* uses the biased variance ($\div m$), while the *running estimate* stores the unbiased one ($\div(m-1)$).
In **eval mode** (`model.eval()`), the batch is ignored and the running buffers are used. BN then becomes a fixed per-channel affine map (Section 9.1).
How many values each statistic pools, for a full batch of 128 images:

where	activation shape	$m = NHW$
stem BN and stage 1	$(128, 16, 32, 32)$	131,072
stage 2	$(128, 32, 16, 16)$	32,768
stage 3	$(128, 64, 8, 8)$	8,192

The last batch of each epoch is smaller (80 images on 50k, 72 on 45k), so its statistics are noisier, but m is still in the thousands.

Detail / deviation: forward hooks and `model.train()`

Because BN updates its running buffers on *every* forward pass in train mode, even under `torch.no_grad()`, a forward pass in the wrong mode changes the model. `scripts/check_eval.py` tests exactly this: it runs 20 no-grad forward passes in train mode and then calls `evaluate`. `evaluate()` and `predict()` both call `model.eval()` first, and `train_one_epoch` calls `model.train()` first, so the training loop always switches modes correctly.

3.7 ReLU

General idea

$\mathrm{ReLU}(z) = \max(0, z)$, element-wise, shape-preserving. Its derivative is $\mathbb{1}[z > 0]$ (PyTorch uses 0 at exactly $z = 0$). It is the only non-linearity in the network.

In this repository

One `nn.ReLU()` module per `BasicBlock` (used twice in `forward`), and one in `ResNetCIFAR` after the stem. Re-using one module object twice is safe because ReLU has no parameters or state. The `inplace` flag is left at its default `False`.

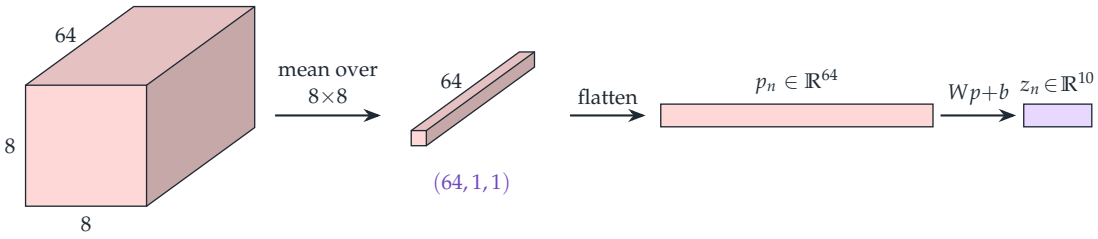
3.8 Global average pooling and flatten

General idea

`nn.AdaptiveAvgPool2d(1)` picks its window so that the output is 1×1. On an 8×8 input that window is the whole map:

$$p[n,c] \;=\; \frac{1}{64}\sum_{h=0}^{7}\sum_{w=0}^{7} A[n,c,h,w], \qquad (N,64,8,8) \;\longrightarrow\; (N,64,1,1).$$

`nn.Flatten(1,-1)` then merges axes 1–3: $(N,64,1,1) \to (N,64)$. It has no parameters and only changes the view.



In this repository

Pooling throws away *where* a feature fired and keeps only *how strongly on average*. That is why the classifier needs just 64×10 weights rather than $64 \cdot 8 \cdot 8 \times 10 = 40{,}960$. There is no max-pooling anywhere in the network: downsampling is done only by stride-2 convolutions.

3.9 Logits, softmax, cross-entropy and class selection

General idea: from scores to a loss

The model outputs *logits* $z \in \mathbb{R}^{10}$ per image: unnormalised scores, any real numbers. Softmax turns them into a probability vector:

$$s_k = \text{softmax}(z)_k = \frac{e^{z_k}}{\sum_{j=0}^{9} e^{z_j}}, \qquad s_k > 0, \quad \sum_k s_k = 1.$$

Adding the same constant to every logit leaves s unchanged ($e^{z_k+c} / \sum e^{z_j+c}$). Libraries exploit this to stay numerically stable, using $\log s_k = z_k - \text{logsumexp}(z)$ with $\text{logsumexp}(z) = M + \log \sum_j e^{z_j - M}$, $M = \max_j z_j$.

The cross-entropy loss for one example with true class y is the negative log-probability of the correct class:

$$\ell(z, y) = -\log s_y = \text{logsumexp}(z) - z_y,$$

and over a batch (PyTorch's default `reduction='mean'`):

$$\mathcal{L} = \frac{1}{N} \sum_{n=1}^{N} \ell(z_n, y_n).$$

If the network is uninformed and outputs equal logits, $s_k = 1/10$ and $\ell = \ln 10 \approx 2.3026$. That is the loss to expect at initialisation.

Class selection. The predicted class is $\hat{y} = \arg\max_k z_k = \arg\max_k s_k$. Softmax is monotonic, so it is not needed to pick the class, only to report a probability.

In this repository: three places that turn logits into decisions

- **Training/eval loss:** `nn.CrossEntropyLoss()` takes *raw logits* $(N, 10)$ and `int64` targets $(N,)$. Internally it is `log_softmax` followed by negative log-likelihood. The model therefore has no softmax layer: putting one in would apply softmax twice.
- **Accuracy:** `torch.argmax(logits, dim=1) == label` in `train_one_epoch` and `evaluate`. `dim=1` is the class axis, giving shape $(N,)$.
- **Probabilities:** `torch.softmax(logits, dim=1)` in `predict()` (evaluation report) and in the service, which then takes `torch.topk(probs, 3)` and returns the three largest (s_k, k) pairs.

4 One residual block in full detail

4.1 The idea: learn a correction, not a whole mapping

General idea: residual learning

A stack of layers is asked to represent some mapping $\mathcal{H}(x)$. A residual block instead computes

$$y = \text{ReLU}\big(\mathcal{F}(x) + s(x)\big),$$

where $\mathcal{F}$ is a small stack of layers (the *residual branch*) and s is a *shortcut* that is the identity whenever shapes allow. The branch only has to learn the difference $\mathcal{F} = \mathcal{H} - x$. If the best thing a block can do is "nothing", it only has to drive $\mathcal{F}$ towards zero, which is easy (shrink the last BN's γ). A plain stack has to learn an exact identity through two ReLU-convolutions, which is hard. He et al. argue that this is why deep plain networks *train* worse (higher training error), not just generalise worse. This repo's ablation reproduces that effect (Section 9.6).

4.2 The code

```
class BasicBlock(nn.Module):
    def __init__(self,in_ch,out_ch,stride,residual):
        super().__init__()
        ...
        self.first_conv = nn.Conv2d(bias=False,in_channels=in_ch,out_channels=out_ch,stride=stride
    ,padding=1,kernel_size=3)
        self.bn1 = nn.BatchNorm2d(out_ch)
        self.relu = nn.ReLU()
        self.second_conv = nn.Conv2d(bias=False,in_channels=out_ch,out_channels=out_ch,stride=1,
    padding=1,kernel_size=3)
        self.bn2 = nn.BatchNorm2d(out_ch)
        self.downsampling_need = False
        if residual:
            if (stride != 1) or (in_ch != out_ch):
                self.downsampling_need = True
    def forward(self,x):
        f = self.first_conv(x)
        f = self.bn1(f)
        f = self.relu(f)
        f = self.second_conv(f)
        f = self.bn2(f)
        if self.residual:
            if self.downsampling_need:
                x = x[:,:,::2,::2]
                pads = (self.out_ch - self.in_ch) // 2
                x = F.pad(x, (0, 0, 0, 0, pads, pads))
            x = x + f
        else:
            x = f
        x = self.relu(x)
        return x
```

4.3 The block as equations

In this repository: exact forward computation of `BasicBlock`

With $\mathcal{W}_1 \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times 3 \times 3}$ (stride S), $\mathcal{W}_2 \in \mathbb{R}^{C_{\text{out}} \times C_{\text{out}} \times 3 \times 3}$ (stride 1) and BN parameters (γ_1, β_1),

$(\gamma_2, \beta_2) \in \mathbb{R}^{C_{\text{out}}}$:

$$
\begin{aligned}
a_1 &= \text{conv}_S(x; \mathcal{W}_1) && \text{first conv: may halve } H, W, \text{ may change depth} \\
b_1 &= \text{BN}_{\gamma_1,\beta_1}(a_1) \\
r_1 &= \text{ReLU}(b_1) \\
a_2 &= \text{conv}_1(r_1; \mathcal{W}_2) && \text{second conv: shape preserved} \\
\mathcal{F}(x) = b_2 &= \text{BN}_{\gamma_2,\beta_2}(a_2) && \text{no ReLU yet}
\end{aligned}
$$

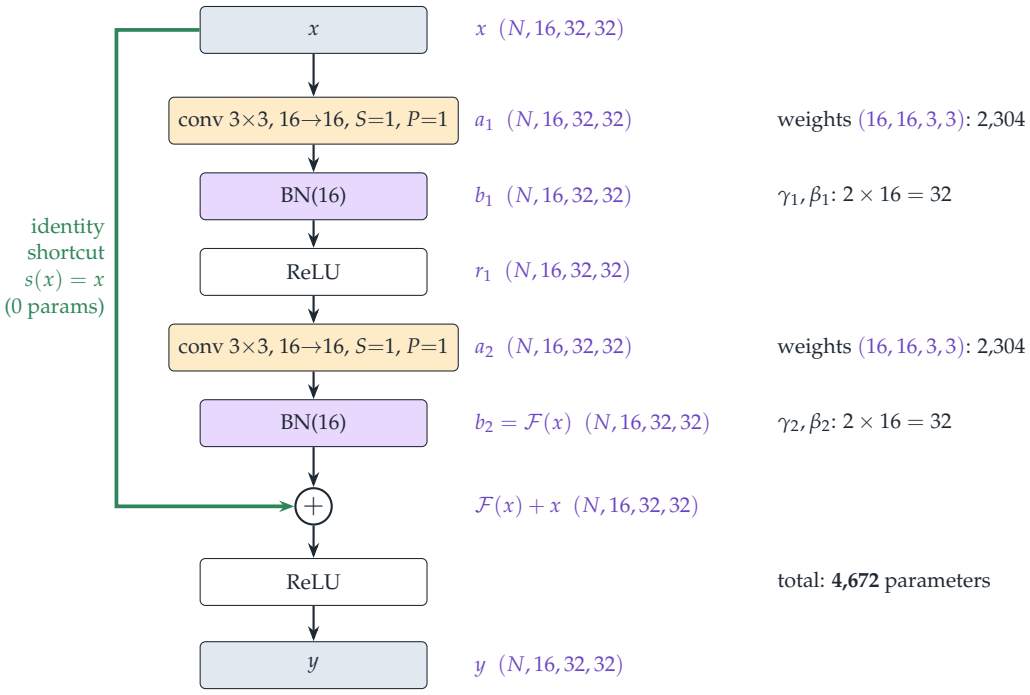
$$
s(x) = \begin{cases} x & \text{residual, } S = 1,\ C_{\text{in}} = C_{\text{out}} \\ \text{pad}_{\text{ch}}\left(x[:, :, :: 2, :: 2]\right) & \text{residual, } S = 2 \text{ or } C_{\text{in}} \neq C_{\text{out}} \\ 0 & \text{plain (}\texttt{residual=False}\text{)} \end{cases}
$$

$$
\boxed{y = \text{ReLU}\left(\mathcal{F}(x) + s(x)\right)}
$$

The ReLU comes *after* the addition. This is the original "post-activation" ResNet v1 block of the 2015 paper, not the 2016 pre-activation variant.

4.4 Identity block: 16 channels in, 16 out, stride 1

This is every block of stage 1, and blocks 2–3 of stages 2 and 3 with their own widths.

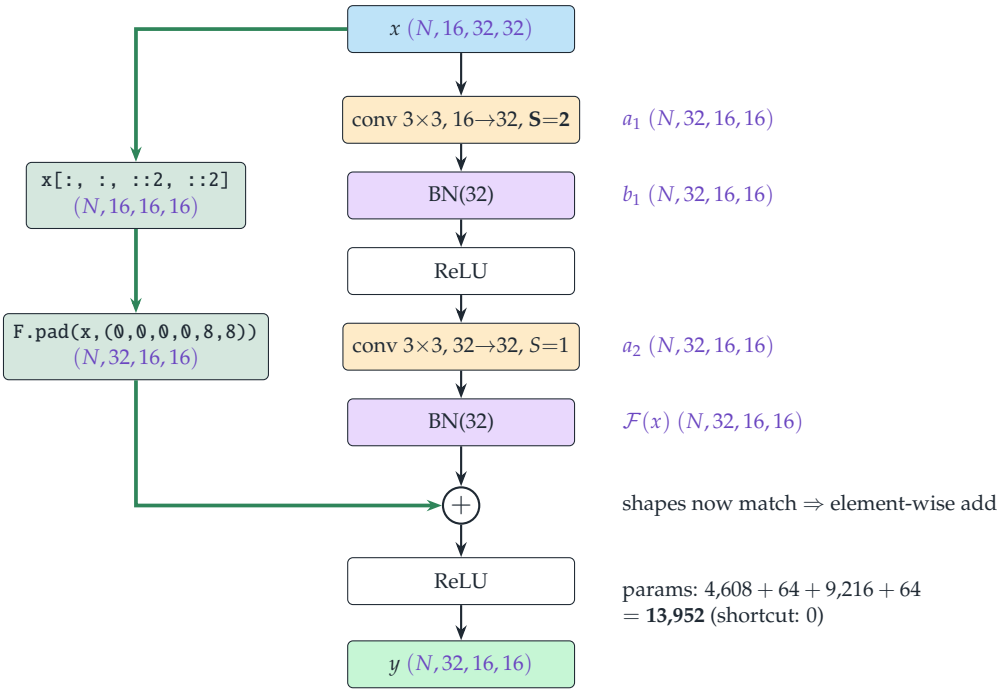


Shape check: identity block, worked

Input $(128, 16, 32, 32)$. First conv: $H_{\text{out}} = \lfloor (32 + 2 - 3)/1 \rfloor + 1 = 32$, depth = number of filters = 16, so $(128, 16, 32, 32)$. BN and ReLU preserve shape. Second conv: same arithmetic, $(128, 16, 32, 32)$. The shortcut is x itself, $(128, 16, 32, 32)$, so the addition is element-wise with no broadcasting (shapes identical). The output is $(128, 16, 32, 32)$. In im2col form, each conv is $(16 \times 144) \cdot (144 \times 1024)$ per image, i.e. 2.36M MACs.

4.5 Downsampling block: 16 channels in, 32 out, stride 2

This is the first block of stage 2 (and, with $32 \to 64$ and $16 \to 8$, the first block of stage 3). Here the two branches would not have the same shape unless the shortcut does something.

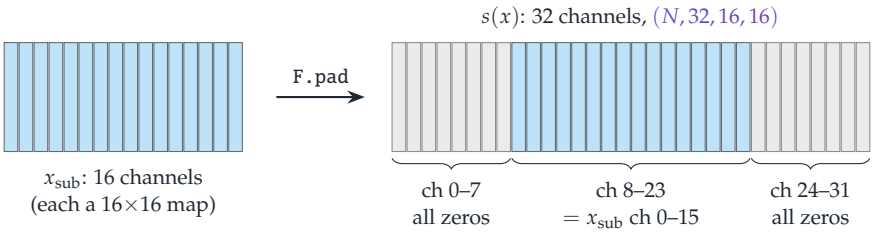


Step 1: spatial subsampling. `x[:,:,::2,::2]` keeps rows $0, 2, 4, \ldots, 30$ and columns $0, 2, \ldots, 30$:

$$x_{\text{sub}}[n, c, i, j] = x[n, c, 2i, 2j], \qquad (N, 16, 32, 32) \to (N, 16, 16, 16).$$

Three quarters of the positions are dropped (no averaging). As shown in Section 3.4, the stride-2 convolution of the main branch centres its windows on the same even positions $(2i, 2j)$, so output pixel (i, j) of the branch and of the shortcut describe the same place in the image.

Step 2: channel zero-padding. `F.pad`'s padding tuple is read *from the last axis backwards* in pairs: (left, right) of W, then (top, bottom) of H, then (front, back) of the channel axis. So `(0,0,0,0,8,8)` leaves W and H alone and adds 8 all-zero channels *before* and 8 *after* the existing 16. Here `pads = (32-16)//2 = 8`; for stage 3 it is `(64-32)//2 = 16`.



$$s(x)[n, c, i, j] = \begin{cases} x[n,\ c-8,\ 2i,\ 2j] & 8 \le c \le 23 \\ 0 & \text{otherwise} \end{cases}$$

$$\Longrightarrow \qquad y[n, c] = \text{ReLU}\left(\mathcal{F}(x)[n, c] + s(x)[n, c]\right).$$

Output channels 0–7 and 24–31 receive *only* the residual branch. Channels 8–23 receive branch + old features.

In this repository: verified on the trained model

Feeding `samples/test0.png` through the trained network and inspecting the first block of stage 2: `s(x)` has shape $(1, 32, 16, 16)$, channels 0–7 and 24–31 are exactly zero, channels 8–23 are not, and recomputing $\mathrm{ReLU}(\mathcal{F}(x) + s(x))$ by hand matches `stage2[0](x)` with maximum difference 0.0.

Detail / deviation: "option A" vs. the alternatives

The paper describes three shortcut options. (A) Zero-padding for increasing dimensions, parameter-free. (B) 1×1 projection convolutions only where dimensions change. (C) Projections everywhere. For the CIFAR-10 networks the paper says "we use identity shortcuts in all cases (i.e., option A)", and this repo follows that, so the shortcut adds **zero parameters** and the plain and residual models have *identical* parameter counts (269,722 each for $n = 3$). The paper does not specify which side the zero channels go on. This repo pads symmetrically (8 before, 8 after), a common choice in public reimplementations; padding only at the end would be equally valid. The subsampling takes every other pixel; it is not average pooling.

4.6 The plain block

In this repository: `residual=False`, used by `-plain`

The branch is identical, but `x = f` replaces `x = x + f`, so $y = \mathrm{ReLU}(\mathcal{F}(x))$. There is no shortcut, `downsampling_need` stays `False`, and the subsample/pad code never runs. Shapes still work because the first convolution alone handles both the stride and the channel change. Same layers, same parameters, same FLOPs: the only difference between "plain-20" and "ResNet-20" is the $+ s(x)$. That is what makes the ablation a clean experiment.

4.7 Why the shortcut helps optimisation (the one-line gradient argument)

General idea

Ignore the final ReLU for a moment, so a block is $y = x + \mathcal{F}(x)$. Its Jacobian is

$$\frac{\partial y}{\partial x} = I + \frac{\partial \mathcal{F}}{\partial x}.$$

Back-propagating through L stacked identity blocks multiplies these:

$$\frac{\partial \mathcal{L}}{\partial x_0} = \frac{\partial \mathcal{L}}{\partial x_L} \prod_{l=1}^{L} \left(I + \frac{\partial \mathcal{F}_l}{\partial x_{l-1}} \right) = \frac{\partial \mathcal{L}}{\partial x_L} \left(I + \sum_{l} \frac{\partial \mathcal{F}_l}{\partial x_{l-1}} + \cdots \right).$$

The expansion always contains the bare term $\partial \mathcal{L}/\partial x_L$, so the gradient reaches early layers unchanged by any weight matrix. In a plain network the gradient is $\prod_l \partial \mathcal{F}_l / \partial x_{l-1}$, a product of L Jacobians, with no such guaranteed path. The real block has a ReLU after the add, which zeroes the gradient where $y = 0$; this weakens the argument but does not remove the path. BN keeps each $\partial \mathcal{F}/\partial x$ well scaled. This is a sketch of the intuition, not a proof. The ablation (plain-56 has *higher training error* than plain-20, while res-56 beats res-20) is the empirical evidence.

4.8 Activations in a real block

In this repository: block `stage1.0` on `samples/test0.png`, trained model, eval mode

tensor	shape	mean	std	min	max
input x (output of stem ReLU)	$(1, 16, 32, 32)$	0.439	0.513	0.00	3.65
a_1 = first conv	$(1, 16, 32, 32)$	−0.325	2.130	−9.15	5.72
b_1 = BN1	$(1, 16, 32, 32)$	0.090	0.806	−3.52	3.29
a_2 = second conv (after ReLU)	$(1, 16, 32, 32)$	0.035	1.493	−8.37	7.26
$\mathcal{F}(x) = b_2$ = BN2	$(1, 16, 32, 32)$	0.044	0.656	−3.52	3.15
$y = \mathrm{ReLU}(\mathcal{F}(x) + x)$	$(1, 16, 32, 32)$	0.615	0.682	0.00	5.31

The convolution outputs have a large spread (std ≈ 2), and BN brings each channel back to std $\lesssim 1$ before the next non-linearity. 25.6% of the block's outputs are exactly zero after the final ReLU. The residual branch $\mathcal{F}(x)$ is small compared with the running signal: its std is 0.66, while the output's mean is 0.62. This is the "small correction" picture of residual learning, measured.

5 The complete ResNet-20 and its shape flow

5.1 The $6n + 2$ family

General idea: counting layers

The CIFAR ResNets of the paper consist of a 3×3 stem convolution, then three *stages* of n basic blocks each, at feature-map sizes 32, 16, 8 with 16, 32, 64 filters, then global average pooling and a 10-way fully connected layer. Each block holds 2 weighted layers, so

$$\underbrace{1}_{\text{stem}} + \underbrace{3 \cdot n \cdot 2}_{\text{blocks}} + \underbrace{1}_{\text{FC}} = 6n + 2 \text{ weighted layers.}$$

$n = 3$ gives 20 and $n = 9$ gives 56. BN, ReLU and pooling are not counted as layers. Only layers with weights are.

In this repository: `ResNetCIFAR(n, residual=True, num_classes=10, in_channels=3, widths=(16,32,64))`

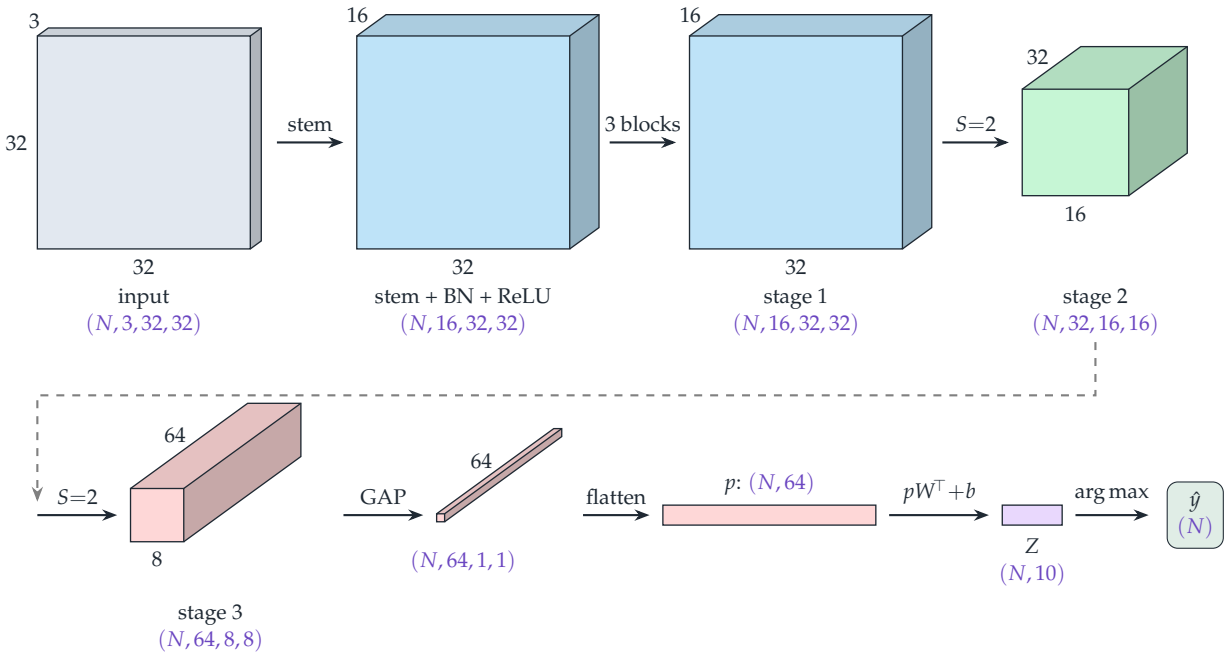
```
        self.stem = nn.Conv2d(bias=False,stride=1,padding=1,kernel_size=3,in_channels=
    in_channels,out_channels=self.widths[0])
        self.bn1 = nn.BatchNorm2d(self.widths[0])
        self.relu = nn.ReLU()
        self.linear = nn.Linear(self.widths[2],self.num_classes)
        self.avg_pool = nn.AdaptiveAvgPool2d(1)
        self.flatten = nn.Flatten(1,-1)
        self.stage1 = self._make_stage(in_ch=widths[0],out_ch=widths[0],stride=1)
        self.stage2 = self._make_stage(in_ch=widths[0],out_ch=widths[1],stride=2)
        self.stage3 = self._make_stage(in_ch=widths[1],out_ch=widths[2],stride=2)
        self._init_weights()
    def _make_stage(self,in_ch,out_ch,stride):
        blocks = []
        blocks.append(BasicBlock(in_ch=in_ch,out_ch=out_ch,stride=stride,residual=self.
    residual))
        for i in range(self.n - 1):
            blocks.append(BasicBlock(in_ch=out_ch,out_ch=out_ch,stride=1,residual=self.
    residual))
```

```
60         return nn.Sequential(*blocks)
```

Only the *first* block of a stage gets the stage's stride and the channel change; the other $n-1$ are identity blocks. Stage 1 has stride 1 and $16 \to 16$, so all its blocks are identity blocks and there is no downsampling at the start. The forward pass is the straight composition

$$z = \text{Linear}\Big(\text{Flatten}\big(\text{GAP}(\text{stage}_3(\text{stage}_2(\text{stage}_1(\text{ReLU}(\text{BN}(\text{stem}(x)))))))\big)\Big).$$

5.2 The shape flow, drawn



Read it as two trends running in opposite directions. The spatial size halves twice ($32 \to 16 \to 8$) and the depth doubles twice ($16 \to 32 \to 64$). The number of values per image shrinks from 16,384 in stage 1 to 8,192 in stage 2 and 4,096 in stage 3: the network trades *where* for *what*.

5.3 The full layer table

#	layer (`state_dict` prefix)	output shape	params	MACs / image	RF
	input	$(N, 3, 32, 32)$	–	–	1
1	`stem` conv 3→16, $S1$	$(N, 16, 32, 32)$	432	442,368	3
	`bn1`, ReLU	$(N, 16, 32, 32)$	32	–	3
2–3	`stage1.0` conv, BN, ReLU, conv, BN, $+x$, ReLU	$(N, 16, 32, 32)$	4,672	4,718,592	7
4–5	`stage1.1`	$(N, 16, 32, 32)$	4,672	4,718,592	11
6–7	`stage1.2`	$(N, 16, 32, 32)$	4,672	4,718,592	15
8–9	`stage2.0` 16→32, $S2$, option-A shortcut	$(N, 32, 16, 16)$	13,952	3,538,944	21
10–11	`stage2.1`	$(N, 32, 16, 16)$	18,560	4,718,592	29
12–13	`stage2.2`	$(N, 32, 16, 16)$	18,560	4,718,592	37
14–15	`stage3.0` 32→64, $S2$, option-A shortcut	$(N, 64, 8, 8)$	55,552	3,538,944	49

#	layer (`state_dict` prefix)	output shape	params	MACs / image	RF
16–17	`stage3.1`	$(N, 64, 8, 8)$	73,984	4,718,592	65
18–19	`stage3.2`	$(N, 64, 8, 8)$	73,984	4,718,592	81
	`avg_pool` (GAP)	$(N, 64, 1, 1)$	0	–	all
	`flatten`	$(N, 64)$	0	–	
20	`linear` 64→10	$(N, 10)$	650	640	
	total		**269,722**	**40,551,040**	

Shape check: how each parameter count is obtained

$$\begin{aligned}
\text{stem} &: 16 \cdot 3 \cdot 3 \cdot 3 = 432, \qquad \text{BN}(16) : 2 \cdot 16 = 32 \\
\text{identity block, width } C &: 2 \cdot (C \cdot C \cdot 9) + 2 \cdot (2C) = 18C^2 + 4C \\
&\Rightarrow \; C{=}16 : 4{,}672, \quad C{=}32 : 18{,}560, \quad C{=}64 : 73{,}984 \\
\text{downsampling block } C \to 2C &: (2C \cdot C \cdot 9) + (2C \cdot 2C \cdot 9) + 2 \cdot (2 \cdot 2C) = 54C^2 + 8C \\
&\Rightarrow \; 16{\to}32 : 13{,}952, \quad 32{\to}64 : 55{,}552 \\
\text{linear} &: 64 \cdot 10 + 10 = 650 \\
\text{total} &: 432 + 32 + 3(4{,}672) + (13{,}952 + 2 \cdot 18{,}560) \\
&\quad + (55{,}552 + 2 \cdot 73{,}984) + 650 \;=\; \mathbf{269{,}722}.
\end{aligned}$$

Per stage: stem 464, stage 1 14,016, stage 2 51,072, stage 3 203,520, linear 650. **Three quarters of all parameters live in stage 3**, because a conv's parameter count grows with C^2 while its spatial size does not enter. Of the 269,722 parameters, 267,696 are conv weights (19 convs), 1,376 are BN γ/β (19 BN layers), and 650 are in the linear layer. The model also carries 1,376 BN running-statistic values (`running_mean`, `running_var`) and 19 `num_batches_tracked` counters as *buffers*: they are saved in the checkpoint (116 `state_dict` entries in total) but not trained by SGD.

The paper reports "0.27M" parameters for ResNet-20 and "0.85M" for ResNet-56; counting the model gives 269,722 and 853,018. The plain networks have exactly the same counts, because the shortcut is parameter-free.

MACs. Every 3×3 conv inside a stage costs 2,359,296 MACs per image, whatever the stage (Section 3.5). The two stride-2 convs cost half that (1,179,648), because they produce a quarter of the pixels with twice the filters of their input. The whole forward pass is ≈ 40.6M MACs per image, ≈ 81M FLOPs, or ≈ 5.2G MACs for a batch of 128. Backward costs roughly twice the forward.

Receptive field (RF). The last column is the side length of the input patch that can influence one output pixel along the *longest* (all-convolution) path. Each 3×3 conv adds 2×(current jump), and the jump doubles at each stride-2 conv: $3, 5, \ldots, 15$ in stage 1 (jump 1), +2 then +4 per conv in stage 2, +4 then +8 in stage 3, ending at $81 > 32$. By stage 3 every unit can see the whole image plus padding, before GAP even averages over all positions. Shortcut paths have smaller receptive fields, so a ResNet behaves like an ensemble of paths of different lengths.

5.4 Initialisation

```
66    def _init_weights(self):
67        for m in self.modules():
68            if isinstance(m,nn.Conv2d):
69                fan_in = m.weight[0].numel()
70                std = (2/fan_in) ** 0.5
71                with torch.no_grad():
72                    m.weight.normal_(0,std)
```

General idea: He (Kaiming) initialisation, derived

Take one output of a conv or linear layer, $y = \sum_{i=1}^{n} w_i x_i$, with n = fan_in inputs, weights i.i.d. with mean 0 and variance σ^2, independent of the inputs. Then

$$\mathrm{Var}(y) = n\,\sigma^2\,\mathbb{E}[x^2].$$

If $x = \mathrm{ReLU}(z)$ with z symmetric around 0, half of the mass is zeroed, so $\mathbb{E}[x^2] = \frac{1}{2}\mathrm{Var}(z)$. Chaining layers,

$$\mathrm{Var}(y_l) = \tfrac{1}{2}\,n_l\,\sigma_l^2\;\mathrm{Var}(y_{l-1}),$$

which neither explodes nor vanishes with depth iff $\frac{1}{2}n_l\sigma_l^2 = 1$:

$$\boxed{\sigma_l = \sqrt{2/n_l}}, \qquad n_l = C_{\text{in}} \cdot K \cdot K.$$

In this repository: what the code initialises, and what it leaves alone

`m.weight[0]` is one filter, shape $(C_{\text{in}}, 3, 3)$, so `numel()` $= 9C_{\text{in}}$ is exactly the fan-in.

conv	fan_in $= 9C_{\text{in}}$	$\sigma = \sqrt{2/\text{fan_in}}$	std of an actual draw (seed 0)
stem (C_{in}=3)	27	0.2722	0.2708
C_{in}=16 (stage 1, first conv of stage 2)	144	0.1179	0.1178 / 0.1172
C_{in}=32 (stage 2, first conv of stage 3)	288	0.0833	0.0832
C_{in}=64 (stage 3)	576	0.0589	0.0586

- **Convolutions**: He-normal with fan-in, overwriting PyTorch's default `kaiming_uniform_(a=`$\sqrt{5}$`)`.
- **Linear layer**: *not* touched, so it keeps PyTorch's default, $W, b \sim \mathcal{U}(-1/\sqrt{64},\, 1/\sqrt{64}) = \mathcal{U}(-0.125, 0.125)$. The README states this explicitly.
- **BatchNorm**: PyTorch default, $\gamma = 1$, $\beta = 0$, running mean 0, running variance 1.

Detail / deviation: fan-in vs. fan-out

He et al. derive both a forward (fan-in) and a backward (fan-out, $n = 9C_{\text{out}}$) condition. This repo uses fan-in. torchvision's reference ImageNet ResNet uses `mode="fan_out"`. With BN after every conv the network is fairly insensitive to the choice, since BN re-normalises each channel on the first forward pass. The stem's input is not a ReLU output, so the factor 2 is not justified there; applying the same rule to every conv is the usual simplification.

5.5 Memory of one forward pass

Per image, the outputs of the main ops are: stem/BN/ReLU $3 \times 16{,}384$; stage 1: 3 blocks $\times$ 7 tensors $\times$ 16,384; stage 2 and 3 likewise with 8,192 and 4,096. That is ≈ 0.65M activation values

per image, about 2.6 MB in `float32`, and on the order of 300 MB for a batch of 128 (autograd saves most, but not exactly all, of them). Autograd keeps these for the backward pass. This, not the 1 MB of parameters, is what fills GPU memory during training, and it is why `evaluate()` wraps the forward in `torch.no_grad()`.

6 A complete forward pass with real numbers

This section follows a single image, `samples/test0.png` (a CIFAR-10 test cat, label 3), through the *trained* model `model/resnet20.pt` in eval mode. Every number below was read from the actual tensors. This is the same image the README uses for its API example, so the final probabilities can be checked against the README.

6.1 Input

In this repository

`Image.open(...).convert("RGB")` gives a 32×32 RGB image, and `np.asarray` turns it into $(32, 32, 3)$ `uint8`. Its top-left pixel is $(158, 112, 49)$ and its centre pixel $(101, 101, 92)$. After `preprocess(arr, mean).unsqueeze(0)`: shape $(1, 3, 32, 32)$, values in $[-0.438, 0.591]$, mean -0.048. The `unsqueeze(0)` adds the batch axis with $N = 1$.

6.2 One output pixel of the stem, computed by hand

The stem convolution has 16 filters of shape $(3, 3, 3)$. Take filter $o = 4$ (the one with the largest norm, $\|\mathcal{W}[4]\|_2 = 3.18$) and output position $(i, j) = (10, 12)$. With $P = 1$, $S = 1$, the window covers padded rows 10..12 and columns 12..14, which are original rows 9..11 and columns 11..13, all inside the image. The output is the sum of the element-wise product of two $3 \times 3 \times 3$ blocks, i.e. a dot product of two 27-vectors:

	input window $\tilde{X}[0, c, 10{:}13, 12{:}15]$	**filter** $\mathcal{W}[4, c]$
$c = 0$ (R)	$\begin{bmatrix} 0.108 & 0.177 & 0.075 \\ 0.140 & 0.214 & 0.107 \\ 0.164 & 0.221 & 0.235 \end{bmatrix}$	$\begin{bmatrix} -0.106 & -0.643 & -1.138 \\ 0.728 & 0.178 & -0.991 \\ 1.029 & 1.219 & 0.467 \end{bmatrix}$
$c = 1$ (G)	$\begin{bmatrix} 0.024 & 0.110 & 0.024 \\ 0.044 & 0.139 & 0.053 \\ 0.077 & 0.151 & 0.190 \end{bmatrix}$	$\begin{bmatrix} 0.177 & 0.360 & 0.357 \\ -0.311 & -0.571 & -0.476 \\ -0.376 & -0.754 & -0.442 \end{bmatrix}$
$c = 2$ (B)	$\begin{bmatrix} 0.006 & 0.108 & 0.029 \\ 0.059 & 0.153 & 0.063 \\ 0.073 & 0.143 & 0.174 \end{bmatrix}$	$\begin{bmatrix} -0.004 & 0.650 & 0.587 \\ -0.394 & 0.289 & 0.604 \\ -0.819 & -0.152 & 0.396 \end{bmatrix}$

$$Y[0, 4, 10, 12] \;=\; \sum_{c=0}^{2} \sum_{u=0}^{2} \sum_{v=0}^{2} \mathcal{W}[4, c, u, v]\, \tilde{X}[0, c, 10 + u, 12 + v] \;=\; 0.21236,$$

which matches PyTorch's `model.stem(x)[0,4,10,12]` $= 0.21236$ to six digits. In the red channel, this filter has a negative top-right and positive bottom-left pattern: a diagonal edge detector.

In this repository: corner pixels use the zero padding

At $(i, j) = (0, 0)$ the window covers padded rows/columns 0..2, and row/column 0 of $\tilde{X}$ is the conv padding (zeros, *not* $-\mu$). Five of the nine positions in each channel are zeros. Only the 2×2 top-left block of real pixels contributes, giving $Y[0, 4, 0, 0] = -0.12623$. Computing it by hand with the padded window again matches PyTorch exactly.

6.3 BatchNorm in eval mode on real parameters

In this repository: stem BN, first four channels

c	γ_c	β_c	r_μ	r_{σ^2}	$a_c = \gamma_c / \sqrt{r_{\sigma^2} + \varepsilon}$	$b_c = \beta_c - a_c r_\mu$
0	0.0112	−0.0338	−0.0038	0.00034	0.6043	−0.0315
1	0.6265	1.4951	−0.0312	0.0937	2.0471	1.5590
2	0.6622	0.3810	−0.0169	0.2595	1.3001	0.4030
3	0.6092	0.0945	−0.1333	0.3837	0.9835	0.2255

In eval mode each channel is just $y = a_c\, x + b_c$ (Section 9.1). Channel 0 is interesting. Its filter has norm 0.047 (all others are 1.2–3.2), its γ is 0.011, and its pre-BN variance is tiny. Its output is therefore ≈ -0.03 everywhere, which the next ReLU turns into 0. Weight decay (Section 8.2) has effectively *switched this channel off*: the trained stem uses 15 of its 16 filters.

6.4 Statistics through the whole network

In this repository: activations of `test0.png`, trained model, eval mode

after	shape	mean	std	max	% zeros
input	(1, 3, 32, 32)	−0.048	–	0.591	–
stem conv	(1, 16, 32, 32)	0.005	0.418	2.12	–
stem BN	(1, 16, 32, 32)	0.356	0.618	3.65	–
stem ReLU	(1, 16, 32, 32)	0.439	0.513	3.65	29.7
stage 1 (3 blocks)	(1, 16, 32, 32)	0.949	0.754	4.83	10.5
stage 2 (3 blocks)	(1, 32, 16, 16)	0.652	0.797	5.66	32.8
stage 3 (3 blocks)	(1, 64, 8, 8)	0.912	1.486	12.05	44.3
GAP + flatten	(1, 64)	0.912	0.802	3.69	0.0
linear (logits)	(1, 10)	0.006	10.25	23.91	–

The activations stay in a narrow, healthy range through 19 convolutions. Normalisation keeps them from exploding, and the shortcuts keep a running signal. Sparsity grows with depth: in stage 3, 44% of the units are silent. The 64 pooled features are all strictly positive (GAP of non-negative maps, min 0.044). The largest are channels 61, 15, 11, 1 and 50.

6.5 From pooled features to a decision

In this repository: the last matrix multiplication, in numbers

$p \in \mathbb{R}^{1\times 64}$, $W^\top \in \mathbb{R}^{64\times 10}$, $b \in \mathbb{R}^{10}$. Logit k is $z_k = \sum_{c=0}^{63} W_{k,c}\, p_c + b_k$:

$$z = (\underbrace{-4.13}_{\text{plane}}, \underbrace{-0.13}_{\text{car}}, \underbrace{0.38}_{\text{bird}}, \underbrace{\mathbf{23.91}}_{\text{cat}}, \underbrace{-10.23}_{\text{deer}}, \underbrace{8.23}_{\text{dog}}, \underbrace{4.10}_{\text{frog}}, \underbrace{-6.30}_{\text{horse}}, \underbrace{-6.56}_{\text{ship}}, \underbrace{-9.22}_{\text{truck}}).$$

The cat logit is the sum of 64 products plus $b_3 = 0.494$. Its three largest terms come from pooled channels 1, 61 and 11 (3.83, 3.73, 2.55). The runner-up is dog, at 8.23.
Softmax: $s_{\text{cat}} = 1/\left(1 + \sum_{k\neq 3} e^{z_k - z_3}\right)$. The dominant term is $e^{8.23-23.91} = e^{-15.68} \approx 1.55 \times 10^{-7}$, so

$$s_{\text{cat}} = 0.99999988, \qquad s_{\text{dog}} = 1.55 \times 10^{-7}, \qquad s_{\text{frog}} = 2.51 \times 10^{-9},$$

which are exactly the three entries the service returns for this image in the README. $\hat{y} = \arg\max_k z_k = 3 = \text{cat}$. The cross-entropy for the true label is $\ell = \text{logsumexp}(z) - z_3 = 1.19 \times 10^{-7}$, essentially zero.

Detail / deviation: a logit gap of 15.7 means overconfidence

A gap that large between the top two logits is typical of this model and is the mechanism behind the overconfidence measured in Section 9.4. Cross-entropy keeps rewarding larger margins on training images even after they are classified correctly, until weight decay balances it. Nothing in the recipe (no label smoothing, no temperature scaling) counteracts this.

6.6 An independent check of every formula in this document

In this repository: NumPy re-implementation

To make sure the equations here describe what the code computes, the forward pass was re-implemented from the formulas of Sections 3 and 4 alone, in `float64` NumPy, reading only the checkpoint's `state_dict`: im2col convolution (Section 3.5), eval-mode BN as $a_c x + b_c$, ReLU, the option-A shortcut (`[:, ::2, ::2]` then 8/16 zero channels on each side), GAP and $Wp + b$. Its logits on `test0.png` match PyTorch's to within 4.5×10^{-6} (the difference is `float32` rounding in PyTorch). The backward formulas of Section 7 were checked the same way against autograd: BN backward to 10^{-15}, conv $\partial/\partial\mathcal{W}$ and $\partial/\partial X$ (both stride 1 and stride 2, via col2im and via transposed convolution) to 10^{-15}, and one SGD step (Section 8.1) against `torch.optim.SGD` to 4×10^{-6}.

7 Back-propagation through every operation

7.1 The rule that organises everything

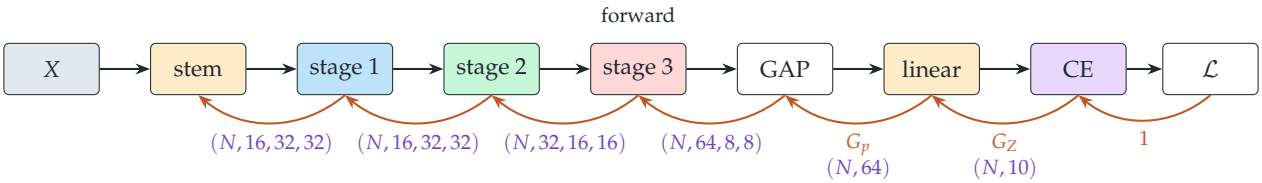
General idea: chain rule, written with shapes

Write $G_T := \partial \mathcal{L} / \partial T$ for any tensor T in the computation. Since $\mathcal{L}$ is a scalar, G_T **always has exactly the shape of** T. For an operation $Y = f(X)$, back-propagation receives G_Y and must produce G_X:

$$G_X[\alpha] \;=\; \sum_{\beta} \frac{\partial Y[\beta]}{\partial X[\alpha]}\, G_Y[\beta] \qquad \big(\text{i.e. } G_X = J_f^{\top} G_Y \text{ with everything flattened}\big).$$

In practice nobody builds J_f, which for one 16→16 conv on a batch would be a 2.1M × 2.1M matrix. Each op gets a hand-derived rule that maps G_Y to G_X (and to G_θ for its parameters). When a tensor is used twice (the block input x feeds both the branch and the shortcut), **its gradients add**.

The backward pass visits the forward graph in reverse:



backward: each arrow carries G of the tensor at that point, same shape as the forward tensor

7.2 Cross-entropy with softmax: the gradient on the logits

General idea: derivation

For one example, $\ell = \operatorname{logsumexp}(z) - z_y$. Since $\partial \operatorname{logsumexp}(z)/\partial z_k = e^{z_k} / \sum_j e^{z_j} = s_k$,

$$\frac{\partial \ell}{\partial z_k} = s_k - \mathbb{1}[k = y].$$

With the batch mean $\mathcal{L} = \frac{1}{N} \sum_n \ell_n$:

$$\boxed{G_Z = \tfrac{1}{N}\,(S - Y_{\text{onehot}})} \qquad S,\ Y_{\text{onehot}},\ G_Z \in \mathbb{R}^{N \times 10}.$$

The gradient on the logits is "predicted probability minus target", divided by the batch size. Each row sums to zero. The correct class is pushed up by $1 - s_y$, and every wrong class is pushed down by its own probability.

In this repository: on `test0.png`, $N = 1$

$G_z = s - e_3 = (6.7{\times}10^{-13},\ 3.6{\times}10^{-11},\ 6.0{\times}10^{-11},\ \mathbf{-1.58{\times}10^{-7}},\ \ldots,\ 1.55{\times}10^{-7}\ (\text{dog}),\ \ldots)$. Almost no gradient: a confidently correct example teaches the model nothing more. During training the batch average is dominated by the hard and misclassified examples.

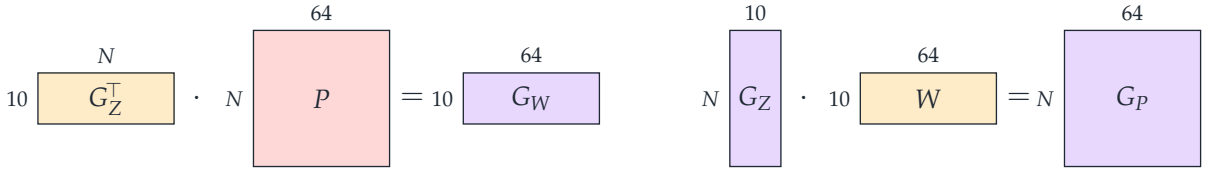
7.3 Linear layer

General idea: three matrix products, each shape-checked

Forward $Z = PW^\top + \mathbf{1}b^\top$ with $P \in \mathbb{R}^{N\times 64}$, $W \in \mathbb{R}^{10\times 64}$. Then

$$\begin{aligned} G_W &= G_Z^\top P && (10\times N)(N\times 64) = (10\times 64)\ \checkmark \text{ same as } W \\ G_b &= G_Z^\top \mathbf{1}_N = \textstyle\sum_n G_Z[n,:] && (10) \\ G_P &= G_Z\, W && (N\times 10)(10\times 64) = (N\times 64)\ \checkmark \text{ same as } P \end{aligned}$$

The mnemonic: *the gradient of a matrix product with respect to one factor is the upstream gradient multiplied by the transpose of the other factor, placed on whichever side makes the shapes fit*. The sum over the batch in G_W and G_b is the inner dimension N disappearing: the parameter gradient accumulates over all examples.



7.4 Flatten and global average pooling

General idea

Flatten backward is a reshape: $G \in \mathbb{R}^{N\times 64} \to \mathbb{R}^{N\times 64\times 1\times 1}$. GAP averaged 64 positions with weight $1/64$ each, so each position receives $1/64$ of the gradient:

$$G_A[n,c,h,w] = \tfrac{1}{64}\, G_p[n,c], \qquad (N,64) \xrightarrow{\text{broadcast}} (N,64,8,8).$$

Every location of a stage-3 feature map gets the same gradient. GAP does not care where the evidence was.

7.5 ReLU

General idea

$G_z = G_y \odot \mathbb{1}[z > 0]$. Gradient flows unchanged through active units and is blocked at inactive ones. Autograd needs to remember only the output (or the mask), not the input.

7.6 The residual addition and the option-A shortcut

General idea: an addition copies the gradient to both inputs

For $u = \mathcal{F}(x) + s(x)$: $\partial u/\partial \mathcal{F} = I$ and $\partial u/\partial s = I$, so $G_{\mathcal{F}} = G_u$ and $G_s = G_u$. Then

$$G_x = \underbrace{(\text{branch backward of } G_u)}_{\text{through 2 convs, 2 BNs, 1 ReLU}} + \underbrace{(\text{shortcut backward of } G_u)}_{\text{identity: just } G_u}.$$

This is the "gradient highway" of Section 4.7, now concretely: the shortcut contributes G_u itself

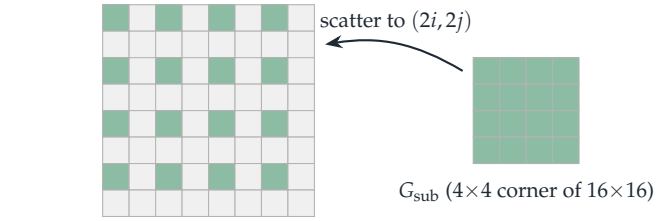
to G_x, untouched by any weight.

In this repository: backward of `F.pad(x[:,:,::2,::2], (0,0,0,0,8,8))`

Run the two forward steps backwards. With $G_s \in \mathbb{R}^{N\times 32\times 16\times 16}$:

1. **Pad backward = slice.** The zero channels did not depend on x, so their gradient is discarded: $G_{\text{sub}} = G_s[:, 8{:}24, :, :] \in \mathbb{R}^{N\times 16\times 16\times 16}$.
2. **Strided slice backward = scatter.** Allocate $G \in \mathbb{R}^{N\times 16\times 32\times 32}$ filled with zeros and write $G[:, :, ::2, ::2] = G_{\text{sub}}$.

So the shortcut sends gradient only to the even-row, even-column positions of the input (a quarter of them) and only from the middle 16 output channels. The other three quarters of the positions get gradient only through the residual branch's stride-2 convolution.



G_x from the shortcut (8×8 corner of 32×32):
green = receives G_{sub}, grey = 0

7.7 Batch normalisation (train mode)

General idea: derivation in four lines

Fix a channel and drop the index c. Write $m = NHW$, $\hat{a}_k = (a_k - \mu)/\sigma_\varepsilon$ with $\sigma_\varepsilon = \sqrt{\sigma^2 + \varepsilon}$, and $y_k = \gamma \hat{a}_k + \beta$, for k running over the m positions. The parameter gradients are immediate:

$$G_\gamma = \sum_{k=1}^{m} G_{y_k}\, \hat{a}_k, \qquad G_\beta = \sum_{k=1}^{m} G_{y_k}.$$

For the input, μ and σ^2 depend on *every* a_k in the channel, so each a_k affects every y_j. Using $\partial\mu/\partial a_k = 1/m$ and $\partial\sigma^2/\partial a_k = 2(a_k - \mu)/m$ and collecting terms:

$$\boxed{G_{a_k} = \frac{\gamma}{m\,\sigma_\varepsilon}\left(m\,G_{y_k} - \sum_j G_{y_j} - \hat{a}_k \sum_j G_{y_j}\hat{a}_j\right)}$$

The upstream gradient is centred (its mean over the channel is removed), and its component along $\hat{a}$ is removed, then rescaled by $\gamma/\sigma_\varepsilon$. Hence a BN layer cannot pass back a gradient that would merely shift or rescale its input channel. Those directions are taken over by β and γ.

Shape check: BN backward

$G_y, G_a \in \mathbb{R}^{N\times C\times H\times W}$. The sums run over axes $(0, 2, 3)$ and give shape (C), broadcast back as $(1, C, 1, 1)$. $G_\gamma, G_\beta \in \mathbb{R}^C$. In eval mode BN is the fixed affine $y = a_c x + b_c$, so its backward would be simply $G_x = a_c\, G_y$. Training never back-propagates in eval mode, though.

7.8 Convolution

General idea: two gradients, both are convolutions again

From $Y[n,o,i,j] = \sum_{c,u,v} \mathcal{W}[o,c,u,v]\ \tilde{X}[n,c,Si+u,Sj+v]$:
Weight gradient: correlate the (padded) input with the output gradient, summing over the batch:

$$G_{\mathcal{W}}[o,c,u,v] = \sum_{n}\sum_{i,j} G_Y[n,o,i,j]\ \tilde{X}[n,c,\ Si+u,\ Sj+v].$$

Input gradient: every input pixel collects gradient from every output window that contained it:

$$G_{\tilde{X}}[n,c,a,b] = \sum_{o} \sum_{\substack{i,j,u,v:\\ Si+u=a,\ Sj+v=b}} G_Y[n,o,i,j]\ \mathcal{W}[o,c,u,v],$$

and G_X is $G_{\tilde{X}}$ with the P-pixel border cropped off. This is a "full" convolution of G_Y (with zeros inserted between entries when $S = 2$) with the 180°-rotated kernel, with the in/out channel roles swapped. It is exactly what `conv_transpose2d` computes.

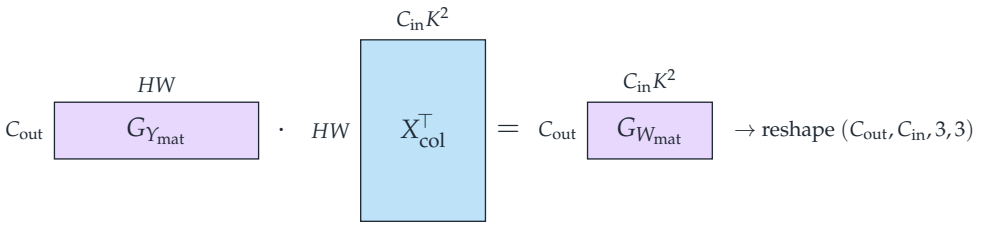
General idea: the same thing in im2col form: two more matrix products

With $Y_{\text{mat}} = W_{\text{mat}} X_{\text{col}}$ (Section 3.5) and $G_{Y_{\text{mat}}} \in \mathbb{R}^{C_{\text{out}} \times H_{\text{out}} W_{\text{out}}}$:

$$G_{W_{\text{mat}}} = \sum_{n} G_{Y_{\text{mat}}}^{(n)}\ X_{\text{col}}^{(n)\top} \qquad (C_{\text{out}} \times HW)(HW \times C_{\text{in}}K^2) = (C_{\text{out}} \times C_{\text{in}}K^2)\ \checkmark$$

$$G_{X_{\text{col}}} = W_{\text{mat}}^{\top}\, G_{Y_{\text{mat}}} \qquad (C_{\text{in}}K^2 \times C_{\text{out}})(C_{\text{out}} \times HW) = (C_{\text{in}}K^2 \times HW)\ \checkmark$$

then *col2im*: scatter-add each column of $G_{X_{\text{col}}}$ back into the $C_{\text{in}} \times K \times K$ window it came from. Overlapping windows add up, which is where the "sum over windows containing the pixel" comes from. These are exactly the linear-layer rules, with X_{col} playing P and W_{mat} playing W.



Shape check: conv gradients in this network

conv	X	Y	$G_{\mathcal{W}}$ (= shape of $\mathcal{W}$)	G_X
stem	$(N,3,32,32)$	$(N,16,32,32)$	$(16,3,3,3)$	(not needed)
stage-1 conv	$(N,16,32,32)$	$(N,16,32,32)$	$(16,16,3,3)$	$(N,16,32,32)$
stage-2 first (S=2)	$(N,16,32,32)$	$(N,32,16,16)$	$(32,16,3,3)$	$(N,16,32,32)$
stage-3 first (S=2)	$(N,32,16,16)$	$(N,64,8,8)$	$(64,32,3,3)$	$(N,32,16,16)$
stage-3 conv	$(N,64,8,8)$	$(N,64,8,8)$	$(64,64,3,3)$	$(N,64,8,8)$

The input image needs no gradient (`requires_grad=False`), so autograd skips G_X for the stem. A stride-2 conv's G_X is back at the *larger* resolution: back-propagation undoes the downsampling.

7.9 One whole block backwards

In this repository: gradient flow through `BasicBlock.forward`, reversed

Given G_y (shape of the block output):

$$
\begin{aligned}
G_u &= G_y \odot \mathbb{1}[u > 0] && u = \mathcal{F}(x) + s(x) \\
G_{b_2} &= G_u && \text{(add: copy to branch)} \\
G_{a_2},\, G_{\gamma_2},\, G_{\beta_2} &= \text{BN-backward}(G_{b_2}) && \text{Section 7.7} \\
G_{r_1},\, G_{\mathcal{W}_2} &= \text{conv-backward}(G_{a_2}) && \text{Section 7.8} \\
G_{b_1} &= G_{r_1} \odot \mathbb{1}[b_1 > 0] \\
G_{a_1},\, G_{\gamma_1},\, G_{\beta_1} &= \text{BN-backward}(G_{b_1}) \\
G_x^{\text{branch}},\, G_{\mathcal{W}_1} &= \text{conv-backward}(G_{a_1}) && \text{(stride } S\text{)}
\end{aligned}
$$

$$
G_x = G_x^{\text{branch}} + \begin{cases} G_u & \text{identity shortcut} \\ \text{scatter}\big(G_u[:,\, p{:}p{+}C_{\text{in}}]\big) & \text{option A} \\ 0 & \text{plain} \end{cases}
$$

where $p = (C_{\text{out}} - C_{\text{in}})/2$ is the number of padded channels on each side (8 or 16). `loss.backward()` does all of this automatically, for all 9 blocks, the stem and the head. Afterwards every parameter's `.grad` holds G_θ with `.grad.shape == param.shape`: this was checked for every parameter, e.g. `stage2.0.first_conv.weight` $(32, 16, 3, 3)$, `linear.weight` $(10, 64)$, `bn1.weight` (16).

Detail / deviation: what `train_step` does around backward

```
4  def train_step(model,x,y,optimizer,loss_fn):
5      optimizer.zero_grad()
6      logits = model(x)
7      loss = loss_fn(logits,y)
8      loss.backward()
9      optimizer.step()
10     return logits.detach(), loss.item()
```

PyTorch *accumulates* into `.grad`, so `zero_grad()` must come first. In PyTorch 2.x it sets `.grad` to `None` (the default `set_to_none=True`) rather than filling with zeros. The returned logits are those of the forward pass *before* the update, detached from the graph, and they are what the epoch's training accuracy is computed from (Section 8.4). `loss.item()` copies the scalar to the CPU, which synchronises with the GPU once per step.

8 Optimisation: SGD, the schedule and the training loop

8.1 The optimiser

In this repository: `scripts/train_resnet.py`

```
56 optim = torch.optim.SGD(params=model.parameters(),lr=0.1,momentum=0.9,weight_decay=1e-4)
57 loss_fct = nn.CrossEntropyLoss()
```

One parameter group containing all 269,722 parameters, with `dampening=0` and

`nesterov=False` (defaults).

General idea: the exact update PyTorch performs

At step t, for every parameter tensor θ (all shapes are the parameter's own shape):

$$g_t = \nabla_\theta \mathcal{L}(\theta_t) + \lambda\,\theta_t \qquad \text{weight decay folded into the gradient}$$

$$v_t = \begin{cases} g_t & t = 1 \\ \mu\, v_{t-1} + g_t & t > 1 \end{cases} \qquad \text{momentum buffer (velocity)}$$

$$\theta_{t+1} = \theta_t - \eta_t\, v_t \qquad \text{step}$$

with $\mu = 0.9$, $\lambda = 10^{-4}$ and η_t from the schedule. The momentum buffer is initialised to the first gradient, not to zero. Unrolled, $v_t = \sum_{k \le t} \mu^{t-k} g_k$ is an exponentially weighted sum of past gradients with an effective memory of $1/(1-\mu) = 10$ steps. Under a constant gradient g, $v \to g/(1-\mu) = 10g$, so **the effective step size is** 10η: 1.0, then 0.1, then 0.01.

Detail / deviation: two momentum conventions

Some papers and frameworks (e.g. Sutskever et al. 2013, Caffe) put the learning rate *inside* the velocity, $v_t = \mu v_{t-1} - \eta_t g_t$, $\theta_{t+1} = \theta_t + v_t$. With a constant η the two are identical ($v^{\text{Caffe}} = -\eta\, v^{\text{PyTorch}}$). They differ only at a learning-rate drop. In PyTorch the stored velocity is unchanged and gets multiplied by the new, smaller η, so the step shrinks $10\times$ immediately. In the other form the old velocity still carries the old η and fades over $\sim$20 steps. Over 16,000-iteration phases the difference is negligible, but it is worth knowing when reading other code.

In this repository: one step verified

Three steps of the update above, written out by hand, reproduce `torch.optim.SGD(lr=0.1, momentum=0.9, weight_decay=1e-4)` to within 4×10^{-6} (`float32`). The optimiser state is one momentum buffer per parameter tensor, with the same shape, i.e. another 269,722 numbers. That is why the training checkpoints (2.2 MB) are twice the size of the exported serving weights (1.1 MB).

8.2 Weight decay

General idea: L2 regularisation, and what it does with BN

Adding $\lambda\theta$ to the gradient is gradient descent on

$$\mathcal{L}(\theta) + \tfrac{\lambda}{2}\|\theta\|_2^2,$$

so with plain SGD, "weight decay" and "L2 penalty" are the same thing. (They differ only for adaptive optimisers like Adam, hence AdamW.) Each step shrinks every weight by the factor $(1 - \eta\lambda)$ before the gradient step: $1 - 10^{-5}$ per step at $\eta = 0.1$.

In a network where every conv is followed by BN, the conv's output is invariant to rescaling its weights ($\text{BN}(\alpha\, \mathcal{W} * x) = \text{BN}(\mathcal{W} * x)$ for $\alpha > 0$). The penalty therefore does not change the function, but it keeps $\|\mathcal{W}\|$ small, and a small $\|\mathcal{W}\|$ makes the same η a *larger relative* step. Weight decay on BN-ed convs acts largely as a control on the effective learning rate.

In this repository: applied to every parameter

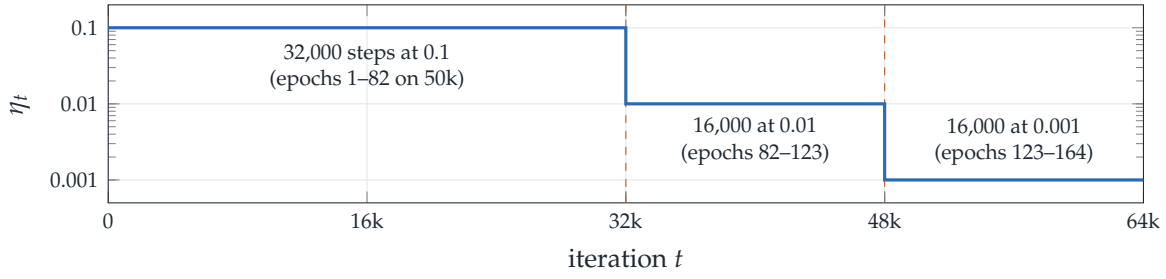
Because `model.parameters()` goes into a single group, decay applies to conv weights, BN γ *and* β, and the linear weight and bias alike. The paper does not say whether BN parameters were excluded; many reimplementations decay everything, as here. Measured on the final model: $\|\theta\|_2^2 = 3218.6$, so the penalty term is $\frac{\lambda}{2}\|\theta\|^2 = 0.161$. At the end of training the data loss is 0.024, so **the regulariser is about** $6.6\times$ **the data term** of the objective SGD is actually minimising. Its visible fingerprint is stem channel 0 (Section 6): its filter norm decayed to 0.047 and its γ to 0.011, a channel the data did not defend.

8.3 The learning-rate schedule

```
60 def lr_at(iteration):
61     if iteration < 32000:
62         return 0.1
63     elif iteration < 48000:
64         return 0.01
65     else:
66         return 0.001
67 def set_lr(optimizer,lr):
68     for p in optimizer.param_groups:
69         p["lr"] = lr
```

General idea: step decay, counted in iterations

$\eta_t = 0.1 \cdot 10^{-[t\geq 32000]-[t\geq 48000]}$, where t is the number of optimiser steps already taken. The paper specifies the schedule in iterations ("divide it by 10 at 32k and 48k iterations, and terminate training at 64k iterations"), independent of the dataset size, which is why the repo counts iterations, not epochs.



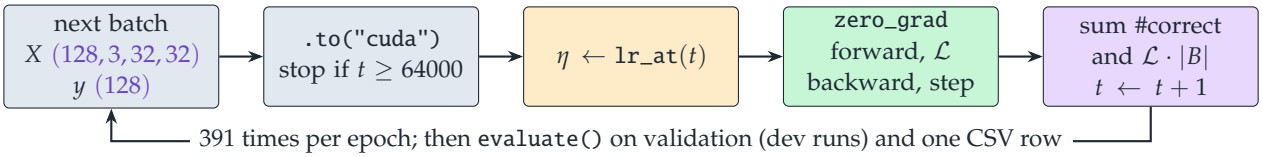
In this repository: iterations ↔ epochs

run	iters/epoch	1st drop (32k)	2nd drop (48k)	end (64k)	last epoch
final (50k)	391	during epoch 82	during epoch 123	epoch 164	267 iterations
development (45k)	352	during epoch 91	during epoch 137	epoch 182	288 iterations

$64{,}000 \times 128 = 8{,}192{,}000$ examples are seen, 163.84 passes over 50k (counting the short last batch of each epoch as a full step, which is what the loop does). $163 \cdot 391 = 63{,}733$ iterations finish 163 epochs, and epoch 164 stops after 267 more, exactly as in `artifacts/logs/final_res20.csv` (last row: epoch 164, iteration 64,000). The learning rate is set *before* every step from the current count, so step 0 through step 31,999 use 0.1 (`check_schedule.py` tests the boundaries 31999/32000 and 47999/48000). Every BN layer's `num_batches_tracked` in the checkpoint reads exactly **64,000**: one train-mode forward per optimiser step, with no extra ones.

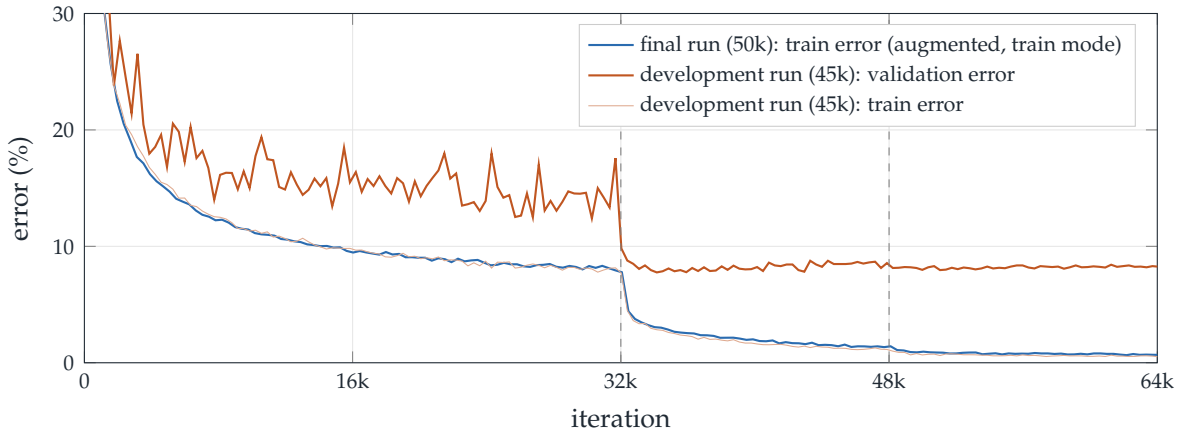
8.4 The training loop

```python
def train_one_epoch(model,loader,optimizer,loss_fn,device,iteration,max_iterations):
    model.train()
    num_exmpl  = 0
    correct_pred = 0
    loss_cum = 0
    iterer = iteration
    for image,label in loader:
        image = image.to(device)
        label = label.to(device)
        if iterer >= max_iterations:
             break
        set_lr(optimizer, lr_at(iterer))
        logits,loss = train_step(model=model,x=image,y=label,optimizer=optimizer,loss_fn=loss_fn)
        compare = (torch.argmax(logits,dim=1) == label)
        compare = compare.sum().item()
        correct_pred+= compare
        iterer+=1
        num_exmpl += logits.shape[0]
        loss_cum+=loss*logits.shape[0]
    loss_cum /= num_exmpl
    correct_pred /= num_exmpl
    return loss_cum,correct_pred,iterer
```



In this repository: what the logged training metrics mean

- **train_loss**: $\sum_{\text{batches}} \mathcal{L}_B\,|B| \,/\, \sum |B|$, the exact per-example average over the epoch, correct also for the short last batch.
- **train_acc**: computed from the logits of each step's forward pass, i.e. on *augmented* images, with *train-mode* BN, and *before* that step's update. The model changes during the epoch, so this is a running average, not the accuracy of the end-of-epoch model. The README's "train error" column (e.g. 0.52% for ResNet-20) is this quantity at the last epoch.
- **val_loss / val_acc**: from `evaluate()` after the epoch, in eval mode, on un-augmented validation images. Empty in the `-full` run, which has no validation set.
- **lr**: read from `optim.param_groups[0]["lr"]` after the epoch, i.e. the rate of its last step.



The two learning-rate drops are the most visible events in training. At the first drop, the development run's validation error falls from 17.6% (epoch 90, $\eta = 0.1$, noisy) to 9.8% one epoch later and 8.8% the epoch after. The second drop gives a smaller, final improvement. At $\eta = 0.1$ the validation curve is noisy because the effective step ($10\eta = 1$) keeps the weights bouncing around a wide region. Smaller steps let SGD settle into it. The final run takes 2,182 s ($\approx$ 36 minutes) on an RTX 3050, about 13 s per epoch.

8.5 Seeds, checkpoints and the run script

In this repository: sources of randomness, and what controls each

randomness	generator	seeded by
weight initialisation	PyTorch global RNG	`torch.manual_seed(args.seed)`, before the model is built
batch order	`torch.Generator g` passed to the loader	`g.manual_seed(args.seed)`
crop offsets and flips	`np.random.default_rng` in the dataset	`seed=args.seed`
train/val split	local `torch.Generator`	literal 0
GPU kernels	cuDNN algorithm choice	*not controlled*

All reported runs use seed 0. Without `torch.backends.cudnn.deterministic=True`, re-running is reproducible up to floating-point noise, not bit-for-bit.

In this repository: checkpoints

`save_checkpoint` writes one dictionary,

```
{"model": state_dict, "optimizer": state_dict, "iteration": t, "seed": s},
```

once, at the end of training. Nothing is saved per epoch, so an interrupted run restarts from scratch. `load_checkpoint` restores the model, restores the optimiser only if one is passed (the service passes `None`), and returns `(iteration, seed)`. `scripts/export_model.py` re-saves the final weights without the optimiser state as `model/resnet20.pt` (1,121,617 bytes, 116 `state_dict` entries) and copies the 50k mean next to it. Those two files are all the service needs.

In this repository: command line

`python -m scripts.train_resnet -name final_res20 -full` for the headline model; `-n 9` for ResNet-56; `-plain` for the plain networks; `-max-iters` to shorten a run. The device is hard-coded to `"cuda"`.

8.6 Sanity checks that were run before the full training

In this repository: scripts/

- `overfit_batch.py`: 200 steps on one fixed batch of 128. A correct model, loss and optimiser must drive this to $\approx$ 100% training accuracy. If they cannot, something is broken before any hyperparameter matters. The first loss should be $\approx \ln 10 = 2.30$. One random batch at initialisation gives 2.42, slightly above because of the randomly initialised linear layer.
- `check_model.py`: parameter counts for $n = 3$ and 9, residual and plain; output shape $(128, 10)$; initial weight standard deviations.
- `check_schedule.py`: `lr_at` on both sides of each boundary.
- `check_checkpoint.py`: train 20 steps, save, load into a fresh model, and verify identical `state_dicts` and identical validation metrics.
- `check_eval.py`: train-mode forward passes followed by `evaluate()`, which must switch BN to eval mode.
- `check_loader.py`, `check_split.py`, `check_mean.py`, `check_augment.py`: data pipeline determinism and shapes, with the augmentation grid saved to `artifacts/aug_grid.png`.

9 Evaluation: eval mode, metrics and results

9.1 What changes in eval mode

General idea: BN collapses to a per-channel affine map

With the running statistics frozen, BN no longer depends on the batch:

$$y = \gamma_c \frac{x - r_{\mu,c}}{\sqrt{r_{\sigma^2,c} + \varepsilon}} + \beta_c = a_c\, x + b_c, \qquad a_c = \frac{\gamma_c}{\sqrt{r_{\sigma^2,c} + \varepsilon}}, \quad b_c = \beta_c - a_c\, r_{\mu,c}.$$

Because the preceding conv is linear, the BN could even be *folded* into it: $\mathcal{W}'[o] = a_o \mathcal{W}[o]$, bias b_o. The repo does not fold. Eval-mode predictions are deterministic and independent of what else is in the batch, so a batch of 1 in the service behaves exactly like a batch of 128 in the evaluation.

In this repository: evaluate() and predict()

Both call `model.eval()` and run inside `torch.no_grad()`, so no graph is stored and the BN buffers are not touched. `evaluate` returns the example-weighted mean loss and the accuracy. `predict` returns `softmax(logits, dim=1)` for every test image as a NumPy array $(10000, 10)$ plus the labels (10000). `eval_test.py` saves both to `artifacts/preds/<name>.npz`, and every metric below is computed from that file. ResNet has no dropout, so BN is the only layer whose behaviour differs between the two modes.

9.2 Confusion matrix

General idea

$C \in \mathbb{N}^{10\times 10}$ with $C[t,p] = \#\{\text{test images of true class } t \text{ predicted as } p\}$. Rows sum to the class sizes (1,000 each), and the trace counts the correct predictions. Accuracy $= \mathrm{tr}(C)/\sum C$.

In this repository: the final ResNet-20 on the 10,000 test images

true \ pred	plane	car	bird	**cat**	deer	**dog**	frog	horse	ship	truck	recall
plane	**923**	4	23	10	5	0	3	1	19	12	0.923
car	7	**966**	0	0	0	0	0	0	2	25	0.966
bird	20	0	**884**	23	24	18	20	5	4	2	0.884
cat	11	0	23	**822**	25	76	21	11	4	7	0.822
deer	2	0	23	14	**927**	12	6	14	0	2	0.927
dog	4	0	14	85	17	**865**	5	8	2	0	0.865
frog	2	1	25	18	8	4	**939**	1	1	1	0.939
horse	5	0	5	16	21	26	2	**922**	2	1	0.922
ship	23	4	4	3	0	0	0	0	**956**	10	0.956
truck	10	22	1	3	0	1	2	1	6	**954**	0.954
precision	.917	.969	.882	.827	.903	.863	.941	.957	.960	.941	

Trace = 9,158, so accuracy = 91.58% and test error = **8.42**%. The two shaded cells, cat→dog (76) and dog→cat (85), are the largest off-diagonal entries, together 161 of the 842 errors. The other classic CIFAR confusions are visible too: car↔truck (25 + 22) and plane↔ship (19 + 23).

9.3 Precision, recall, F1

General idea: row and column sums of C

For class k: $TP_k = C[k,k]$, $FP_k = \sum_t C[t,k] - TP_k$ (column), $FN_k = \sum_p C[k,p] - TP_k$ (row). Then

$$\text{precision}_k = \frac{TP_k}{TP_k + FP_k}, \qquad \text{recall}_k = \frac{TP_k}{TP_k + FN_k}, \qquad F1_k = \frac{2\,\text{prec}_k\,\text{rec}_k}{\text{prec}_k + \text{rec}_k}.$$

Worked for cat: $TP = 822$, the cat column sums to 994, so precision $= 822/994 = 0.827$; the cat row sums to 1000, so recall $= 0.822$; $F1 = 2(0.827)(0.822)/(1.649) = 0.824$.

In this repository

`per_class_metrics(C)` implements exactly this with `np.diag`, `C.sum(axis=0)` (columns) and `C.sum(axis=1)` (rows), and uses `np.divide(..., where=...)` to return 0 instead of dividing by zero for a class that is never predicted. The mean F1 is 0.9158. It equals the accuracy here only because every class has exactly 1,000 test images and precision ≈ recall per class.

9.4 Calibration and expected calibration error

General idea: does a 0.9 confidence mean 90% correct?

For each test image let $\text{conf}_i = \max_k s_{ik}$ and $\text{correct}_i = [\hat{y}_i = y_i]$. Split $(0,1]$ into $B = 15$ equal

bins $I_b = (\frac{b-1}{15}, \frac{b}{15}]$ and let $\mathcal{B}_b = \{i : \text{conf}_i \in I_b\}$. Then

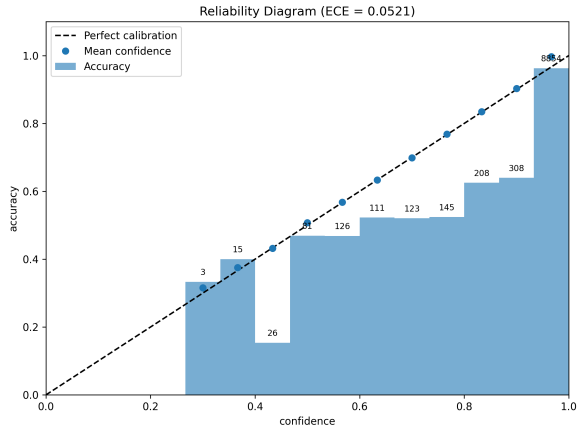
$$\text{ECE} = \sum_{b=1}^{15} \frac{|\mathcal{B}_b|}{n} \Big| \underbrace{\tfrac{1}{|\mathcal{B}_b|} \textstyle\sum_{i \in \mathcal{B}_b} \text{correct}_i}_{\text{acc}(b)} - \underbrace{\tfrac{1}{|\mathcal{B}_b|} \textstyle\sum_{i \in \mathcal{B}_b} \text{conf}_i}_{\text{conf}(b)} \Big|.$$

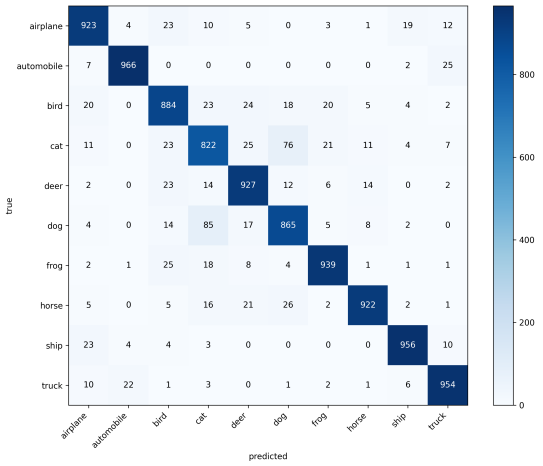
A perfectly calibrated model has $\text{acc}(b) = \text{conf}(b)$ in every bin and ECE $= 0$.

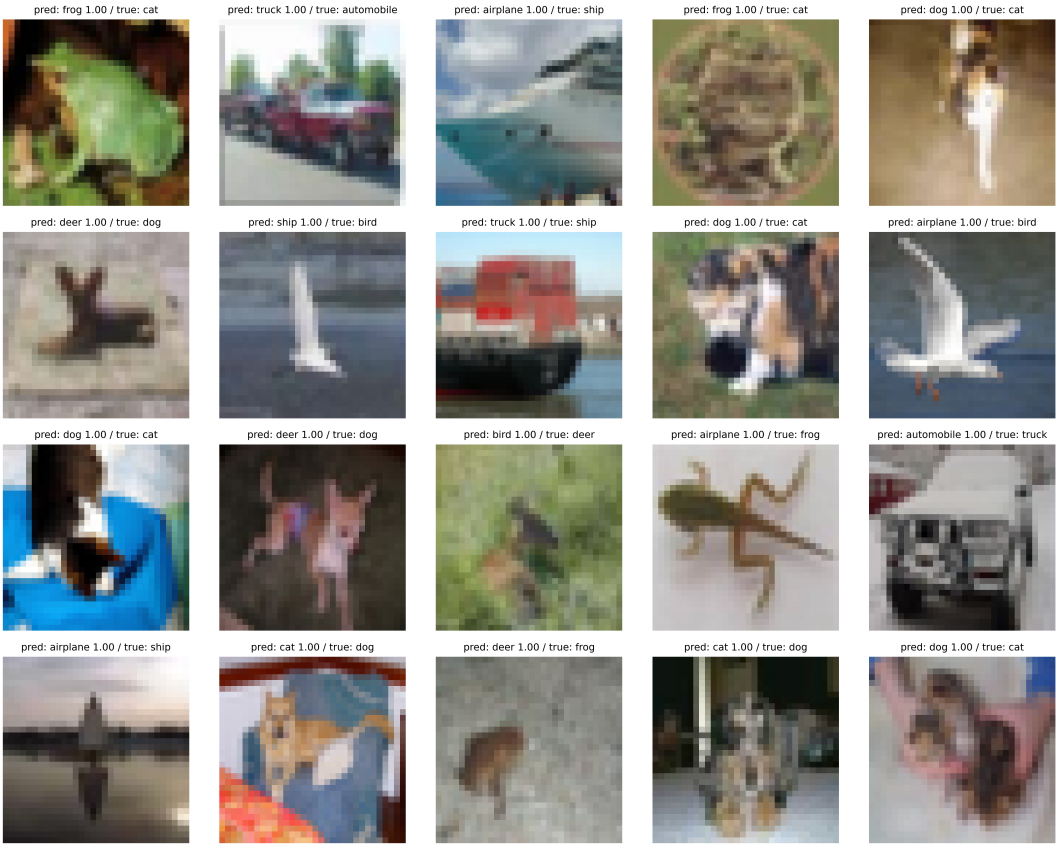
In this repository: `calibration(probs, labels, n_bins=15)`

bin upper edge	$\le \frac{4}{15}$	$\frac{5}{15}$	$\frac{6}{15}$	$\frac{7}{15}$	$\frac{8}{15}$	$\frac{9}{15}$	$\frac{10}{15}$	$\frac{11}{15}$	$\frac{12}{15}$	$\frac{13}{15}$	$\frac{14}{15}$ / 1
$\lvert\mathcal{B}_b\rvert$	0	3	15	26	81	126	111	123	145	208	308 / **8854**
$\text{acc}(b)$	–	.333	.400	.154	.469	.468	.523	.520	.524	.625	.640 / .963
$\text{conf}(b)$	–	.316	.375	.432	.508	.568	.634	.699	.768	.835	.903 / .997

ECE $= 0.0521$. The mean confidence is 0.968 against an accuracy of 0.916. In almost every bin confidence exceeds accuracy, so the model is **overconfident**. 88.5% of all predictions land in the top bin, where the model claims 99.7% and is right 96.3% of the time. That single bin contributes $0.885 \times 0.034 = 0.030$ of the ECE. Bins are half-open $(lo, hi]$ via `(conf > lo) & (conf <= hi)`. A confidence of exactly 0 would be missed, but it is impossible here since $\max_k s_k \ge 0.1$.







The 20 most confident wrong predictions (`docs/figures/confident_errors.png`). Six are cat↔dog swaps, and the first is plausibly a label error in the dataset.

9.5 Headline result

In this repository

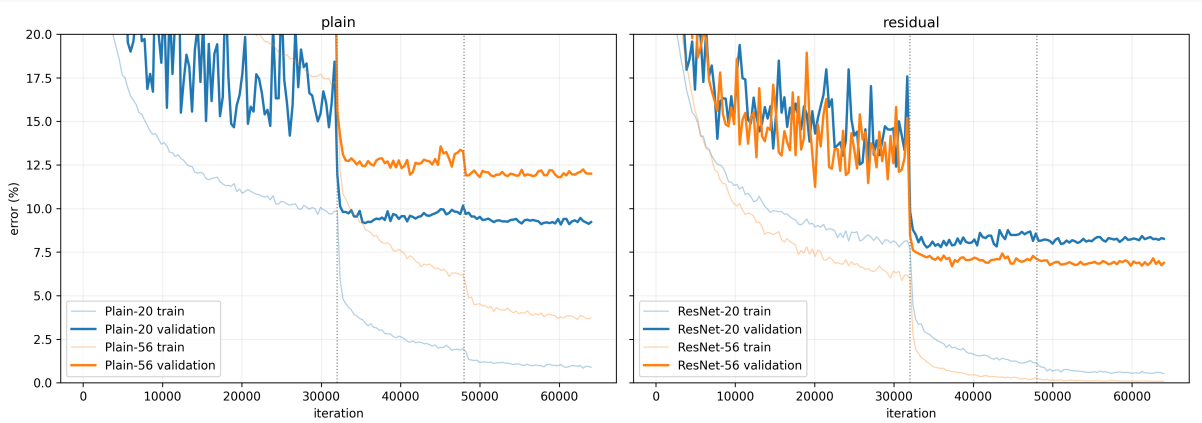
model	test error, this repo	test error, paper
ResNet-20 (trained on 50k, 64k iterations)	8.42%	8.75%

Both are single runs. The paper's 5-run spread for ResNet-110 is ±0.16 points, so a 0.33-point gap is "matches the paper", not "beats it". The test set was used once, after all choices had been made on the 45k/5k split.

9.6 Depth ablation: plain vs. residual

In this repository: same recipe, 45k training images, seed 0

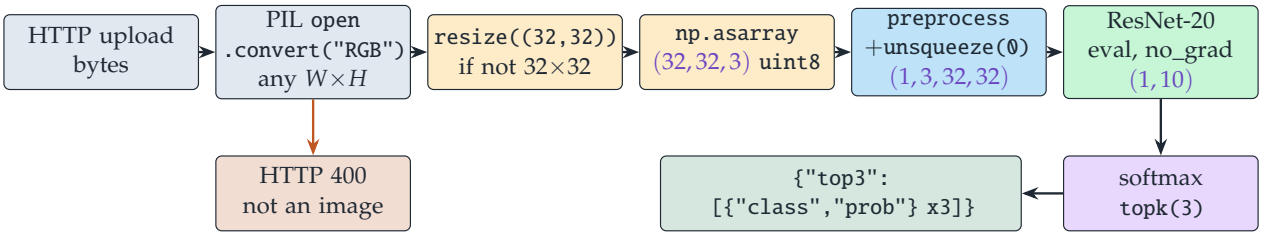
run	depth	type	train error	val error	test error
ResNet-20	20	residual	0.52%	8.26%	9.29%
plain-20	20	plain	0.88%	9.22%	9.77%
ResNet-56	56	residual	0.08%	6.88%	**7.92%**
plain-56	56	plain	**3.72%**	12.00%	12.94%



Going from 20 to 56 layers makes the plain network *worse* (test 9.77 $\rightarrow$ 12.94%), and its *training* error rises too (0.88 $\rightarrow$ 3.72%). The deeper plain net is not overfitting. It fails to optimise, which is the degradation problem the paper set out to solve. The residual network improves with depth (9.29 $\rightarrow$ 7.92%). The two pairs have identical parameters and FLOPs (853,018 for both 56-layer nets), so the shortcut connection is the only difference. ResNet-56 takes 5,171 s ($\approx$ 86 min) vs. 2,238 s for ResNet-20 on the same GPU.

10 Inference and serving

```
model = ResNetCIFAR(3)
load_checkpoint("model/resnet20.pt", model, None, device)
model.eval()
mean = np.load("model/pixel_mean_50k.npy")
...
@app.post("/predict")
async def predict_image(file: UploadFile):
    raw = await file.read()
    try:
        img = Image.open(BytesIO(raw)).convert("RGB")
    except UnidentifiedImageError:
        raise HTTPException(status_code=400,detail="Uploaded file is not a valid image")
    if img.size != (32,32):
        img = img.resize((32,32))
    arr = np.asarray(img)
    x = preprocess(arr,mean=mean).unsqueeze(0)
    with torch.no_grad():
        probs = torch.softmax(model(x),dim=1)
        values,indices = torch.topk(probs,3)
```



In this repository: design points, each with a reason

- **Same preprocess as training.** The service imports it from `cifar_resnet/data.py` and uses the 50k mean that the final model was trained with. Training–serving skew in normalisation is the most common silent deployment bug, and sharing the function rules it out.
- **Loaded once, at import time**, in eval mode on CPU. `load_checkpoint(..., None, ...)` skips the optimiser. The app prints the parameter count (269,722) and `model.training` (`False`) at start-up as a self-check.
- **Any image size is accepted** and resized to 32×32 with Pillow's default resampling filter. A large photo loses most of its detail, so predictions on it are less reliable than on CIFAR-like images. The README says so.
- **`.convert("RGB")`** turns grayscale, palette and RGBA images into three channels. An alpha channel is dropped, not composited.
- **Non-images** raise `UnidentifiedImageError` in PIL, which is mapped to HTTP 400 with a clear message instead of a 500.
- **Batch of one**: `unsqueeze(0)` adds $N = 1$. This is safe only because BN is in eval mode. In train mode a batch of one would make σ_c^2 come from a single image.
- **`topk(probs, 3)`** returns the 3 largest values and their indices along the last axis, both $(1, 3)$, already sorted in decreasing order. The test `test_valid_image` checks the order and that `test0.png` is a cat.

In this repository: latency, from the README

measurement (100 requests after 10 warm-up, 32×32 PNG)	p50	p95
native on Windows (uvicorn, CPU)	3.9 ms	7.0 ms
Docker Desktop on Windows (CPU)	48 ms	50 ms
model forward pass alone	1.4 ms	–

At ≈ 40.6M MACs, the forward pass takes about 1.4 ms on a CPU. The rest of the native request time is HTTP, multipart parsing and PIL decoding. The extra ≈ 44 ms in Docker Desktop is in the request path into the container, not in the model.

In this repository: container

`Dockerfile`: `python:3.13-slim`, CPU-only `torch==2.14.1` from the PyTorch CPU index, then `requirements.txt` (FastAPI, NumPy, Pillow, python-multipart, uvicorn). It copies only `cifar_resnet/`, `service/` and `model/` and runs `uvicorn service.app:app -host 0.0.0.0 -port 8000`. No torchvision is needed at serving time: `data.py` imports only `torch` and `numpy`.

A One-page cheat sheet

Forward shapes for a training batch ($N = 128$)

step	operation	shape
raw	`data.data[idx]`	$(32, 32, 3)$ `uint8`
augment	pad 4 $\rightarrow$ $(40, 40, 3)$, crop, flip	$(32, 32, 3)$ `uint8`
preprocess	$/255$, $-\mu$, permute	$(3, 32, 32)$ `float32`
batch	stack	$(128, 3, 32, 32)$
stem	conv 3$\rightarrow$16 ($3\times3, S1, P1$) + BN + ReLU	$(128, 16, 32, 32)$
stage 1	3 identity blocks	$(128, 16, 32, 32)$
stage 2	block 16$\rightarrow$32, $S2$ + 2 identity blocks	$(128, 32, 16, 16)$
stage 3	block 32$\rightarrow$64, $S2$ + 2 identity blocks	$(128, 64, 8, 8)$
GAP	mean over 8×8	$(128, 64, 1, 1)$
flatten		$(128, 64)$
linear	$pW^\top + b$, W $(10, 64)$	$(128, 10)$
loss	CE, mean	scalar

Formulas

conv output size:	$H_{\text{out}} = \lfloor (H + 2P - K)/S \rfloor + 1 \quad (32 \rightarrow 32$ at S=1, $32 \rightarrow 16$ at S=2)
conv params / MACs:	$C_{\text{out}}C_{\text{in}}K^2$ / $C_{\text{out}}C_{\text{in}}K^2H_{\text{out}}W_{\text{out}}$
im2col:	$Y_{\text{mat}} = W_{\text{mat}}X_{\text{col}}$, $(C_{\text{out}} \times C_{\text{in}}K^2)(C_{\text{in}}K^2 \times H_{\text{out}}W_{\text{out}})$
BN (train):	$y = \gamma\,(x - \mu_B)/\sqrt{\sigma_B^2 + \varepsilon} + \beta$, stats over (N, H, W), $\varepsilon = 10^{-5}$
BN running stats:	$r \leftarrow 0.9\,r + 0.1\,\text{stat}_B \quad$ (unbiased variance)
block:	$y = \text{ReLU}\left(\text{BN}(\text{conv}(\text{ReLU}(\text{BN}(\text{conv}_S(x))))) + s(x)\right)$
option-A shortcut:	$s(x) = \text{pad}_{\text{ch}}\left(x[:, :, :: 2, :: 2]\right)$, $(C_{\text{out}} - C_{\text{in}})/2$ zero channels each side
He init:	$\mathcal{W} \sim \mathcal{N}(0,\, 2/(9C_{\text{in}}))$
softmax, CE:	$s = \text{softmax}(z), \quad \ell = \text{logsumexp}(z) - z_y$
logit gradient:	$G_Z = (S - Y_{\text{onehot}})/N$
linear gradients:	$G_W = G_Z^\top P, \quad G_b = \sum_n G_Z[n], \quad G_P = G_Z W$
BN gradient:	$G_a = \frac{\gamma}{m\sigma_\varepsilon}\left(mG_y - \sum G_y - \hat{a}\sum G_y\hat{a}\right)$
conv gradients:	$G_{W_{\text{mat}}} = \sum_n G_{Y_{\text{mat}}}X_{\text{col}}^\top, \quad G_{X_{\text{col}}} = W_{\text{mat}}^\top G_{Y_{\text{mat}}}$ (then col2im)
SGD:	$v \leftarrow 0.9\,v + (\nabla\mathcal{L} + 10^{-4}\theta), \quad \theta \leftarrow \theta - \eta v$
schedule:	$\eta = 0.1$ $(t < 32\text{k})$, 0.01 $(t < 48\text{k})$, 0.001 $(t < 64\text{k})$
ECE:	$\sum_b \frac{\lvert\mathcal{B}_b\rvert}{n}\lvert\text{acc}(b) - \text{conf}(b)\rvert$, 15 bins

Numbers worth remembering

parameters (ResNet-20)	269,722	parameters (ResNet-56)	853,018
share of params in stage 3	75%	MACs / image	40.6M
iterations / epoch (50k)	391	epochs (final run)	164
training time (RTX 3050)	36 min	test error	8.42% (paper 8.75%)
ECE (15 bins)	0.052	mean confidence / accuracy	0.968 / 0.916
CPU latency p50 (native)	3.9 ms	forward pass alone	1.4 ms

B Things this project does *not* contain

So that future-me does not "remember" features that were never there:

- No 1×1 projection shortcuts (options B/C), no bottleneck blocks, no pre-activation blocks.
- No max-pooling, no dropout, no hidden fully connected layers, no softmax layer inside the model.
- No division by a pixel standard deviation, no colour jitter, no Cutout/Mixup, no label smoothing.
- No learning-rate warm-up (the paper uses one only for ResNet-110), no cosine schedule, no Nesterov momentum.
- No exclusion of BN/bias parameters from weight decay.
- No test-time augmentation: the test set is evaluated on single, original 32×32 views.
- No temperature scaling or other post-hoc calibration.
- No mixed precision, no multi-GPU, no data-loader workers, no per-epoch checkpointing.

C How this document was verified

Every code excerpt is copied from the repository at commit `bf47649` ("README fixes before going public"). Every number labelled as measured was produced by loading `model/resnet20.pt` and `model/pixel_mean_50k.npy` into `ResNetCIFAR(3)` from `cifar_resnet/model.py`, by reading `artifacts/logs/*.csv` and `artifacts/preds/final_res20.npz`, or by counting parameters of the model classes. The forward and backward formulas were checked numerically against PyTorch (Section 6). This document was drafted with the help of an AI assistant from the repository's code and artifacts. The code it describes was written by hand.